FIELD MANUAL Â· CI/CD FROM ZERO

The Kanban Pipeline Manual

Everything we actually did to take a Next.js Kanban board from a folder on a Windows laptop to a Docker image built by GitHub Actions, stored in Amazon ECR, and run on an EC2 server â€” written for someone who has never touched DevOps.

App Next.js 16 Â· MongoDB Â· port 3000 **Repo** Nasir1504/kanban-board **Cloud** AWS us-east-1
Compiled 2 Sep 2026

This manual is assembled from our real conversations, in the order a beginner should learn them â€” not the order they happened. Every command and configuration here is one we actually discussed. Nothing is invented.

How to read the status tags

`IMPLEMENTED` we built this and it works `EXPLAINED` background added so the above makes sense
`NOT DONE YET` we discussed it; it isn't built

CONTENTS

01 What CI/CD actually means

Two letters, two ideas, and one habit: never ship anything a machine hasn't checked first.

Continuous Integration (CI)

CI is the machine that watches your repository. Every time code arrives, it independently downloads a fresh copy, installs the dependencies, and runs your checks â€” formatting, linting, tests, a build. If any check fails, you find out in minutes instead of when a user does.

The word *integration* is historical: before CI, developers worked alone for weeks and then merged everything at once, which was agony. "Continuous" integration means merging small pieces constantly, with a robot verifying each one.

Continuous Delivery / Deployment (CD)

CD is what happens after the checks pass. It has two commonly used meanings, and people mix them up:

TERM	MEANING	OURS
Continuous Delivery	Every good commit is automatically packaged into a ready-to-release artifact. A human clicks the last button.	This is where we are today
Continuous Deployment	Every good commit goes all the way to the live server with no human involved.	The next step

WHERE WE DREW THE LINE

In our project, the *artifact* is a Docker image, and the shelf it sits on is Amazon ECR. Our pipeline builds it and puts it on the shelf automatically. Taking it off the shelf and running it on the server is still done by hand. That is textbook continuous *delivery*.

The mental model that survives every tool

Jenkins, GitHub Actions, GitLab CI, CircleCI â€” they all differ in syntax and nothing else. Every CI/CD system answers exactly four questions:

#	QUESTION	OUR ANSWER
1	**When** do I run? (the trigger)	Push to `main`, or a pull request targeting `main`
2	**What** must pass before I proceed? (the gates)	Prettier format check, ESLint, a successful Docker build
3	**What** artifact do I produce? (the build)	A Docker image tagged with the commit SHA and `latest`
4	**How** does it reach production, and how do I undo it? (deploy + rollback)	Manual pull on EC2; rollback = run the previous SHA tag

Once you have built one pipeline properly, learning a second tool is a syntax exercise â€” roughly a day of work.

Why bother, concretely

- **Broken code cannot reach main.** The pull-request check is a gate, not a suggestion.
- **Builds become reproducible.** The image built by the robot is identical every time; "works on my machine" stops being a defence.
- **Deploys become boring.** Boring is the goal. Exciting deploys are outages.
- **Rollback becomes possible.** Because every image is tagged with its commit SHA, going back is one command.

CHECK YOURSELF Â· PART 1

1. In your own words, what is the difference between continuous delivery and continuous deployment â€” and which one does our project do?
2. Our pipeline runs Prettier and ESLint before it builds the Docker image. Which of the four questions does that answer?
3. If a teammate says "CI is green", what has actually been proven about the code?

02 Our application architecture

Before any pipeline makes sense, you need a clear picture of the pieces and where each one physically lives.

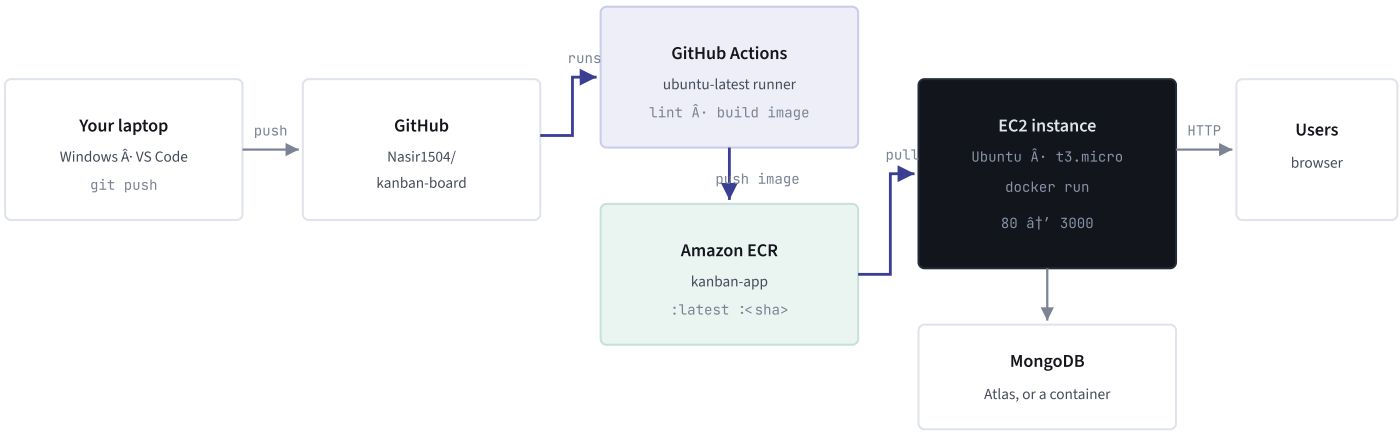
The application itself

PIECE	WHAT IT IS	DETAIL
Next.js 16	The web framework â€” renders the Kanban board	Built with `output: "standalone"`
React 19 + dnd-kit	The UI and the drag-and-drop cards	â€”
MongoDB via Mongoose	Stores boards and tasks	Connection string in `MONGODB_URI`
Port 3000	Where the server listens inside the container	`EXPOSE 3000` in the Dockerfile

WHY OUTPUT: "STANDALONE" MATTERS

Normally a Next.js app needs the whole `node_modules` folder to run. With `standalone`, Next copies only the files actually needed into `.next/standalone`, including a small `server.js`. That folder is self-contained, so our final Docker image can be a few hundred megabytes instead of well over a gigabyte. Our Dockerfile copies exactly that folder â€” **if you ever remove this line from `next.config.mjs`, the Docker build breaks**, because `.next/standalone` will not exist.

Where everything lives



THE WHOLE SYSTEM. BLUE ARROWS ARE THE PART WE AUTOMATED.

The real identifiers we used

THING	VALUE
GitHub repository	`Nasir1504/kanban-board` `NOTE THE FOLDER ON DISK IS KANBAN-APP`
AWS region	`us-east-1` (N. Virginia)
AWS account ID	`239068269281`
ECR repository	`kanban-app`
Full image address	`239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app`
IAM role for CI	`github-actions-ecr-push`
SSH key file	`nasir-ec2-server.pem`

A TRAP WE WALKED INTO

The *folder* on your laptop is called `kanban-app`, the *ECR repository* is called `kanban-app`, but the *GitHub repository* is called `kanban-board`. Three names, two of which match, and one that doesn't. This cost us real debugging time in Part 12. When something says "repo", always check *which* repo.

CHECK YOURSELF Â· PART 2

1. What breaks if you delete `output: "standalone"` from `next.config.mjs`, and at which stage would you notice?
2. The container listens on port 3000, but users reach the site on port 80. Which piece of the system translates between them?
3. Write out the full image address for our app from memory. Which three pieces of information does it contain?

03 Git & GitHub setup

CI is triggered by Git events, so the branching habit you adopt *is* your pipeline design.

STEP 3.1 IMPLEMENTED

Work on a branch, merge through a pull request

PURPOSE

To keep `main` always deployable. Anything on `main` is, by definition, something the robot approved.

WHAT WE DID

Every change went onto a feature branch, then into a pull request against `main`, then merged. Our history shows the pattern repeatedly â€” for example `feat/Update-Docker-configuration` â†’ PR #5 â†’ merged into `main`.

COMMANDS

```
# start a branch
git checkout -b feat/Update-Docker-configuration

# stage, commit, push
git add .github/workflows/ci.yml
git commit -m "ci: push image to ECR on main"
git push -u origin feat/Update-Docker-configuration

# open the pull request (GitHub CLI)
gh pr create --base main --title "Push image to ECR from CI" --fill
```

PLAIN ENGLISH

`git add` chooses which changed files to include. `git commit` saves them as one labelled snapshot on your machine. `git push` uploads that snapshot to GitHub. A **pull request** is a request to copy your branch's commits into `main` â€” it is a review page, not a command.

EXPECTED

Pushing a *branch* runs nothing. Opening the *pull request* starts a CI run. Clicking **Merge** puts a commit on `main`, which starts a second, fuller CI run.

PROBLEM

We pushed a branch and expected CI to run. Nothing happened. Our workflow triggers on `push` to `main` and on `pull_request` targeting `main` â€” a plain branch push matches neither. Opening the PR fixed it.

The two-run pattern, and why it matters

MOMENT	WHAT CI DOES	WHY
You open a PR	Format check, lint, `docker build`. **No push to ECR.**	The code isn't approved yet. Publishing it would pollute the registry and overwrite the `latest` tag with unreviewed code.
The PR is merged	Everything above, **plus** log in to AWS and push the image	A merge *is* a push to `main`. That is the moment code is blessed.

Test it on the PR, ship it on the merge. That single sentence explains most of our workflow file.

GitHub Secrets vs Variables

	SECRETS	VARIABLES
Where	Repo â†’ Settings â†’ Secrets and variables â†’ Actions	
Readable after saving?	No, write-only	Yes
Shown in logs?	Masked as `***`	Printed plainly
Referenced as	`${{ secrets.NAME }}`	`${{ vars.NAME }}`
Use for	Passwords, tokens, private keys	Region names, ARNs, repo names

We used **Variables** for all three of our AWS values, because with OIDC (Part 12) *none of them are secret*:

NAME	VALUE
`AWS_REGION`	`us-east-1`
`AWS_ROLE_ARN`	`arn:aws:iam::239068269281:role/github-actions-ecr-push`
`ECR_REPOSITORY`	`kanban-app`

TWO MISTAKES WE MADE ON THIS SCREEN

1. Copying names out of a formatted table brought a **trailing space** along, and GitHub rejected it with "Variable names may only contain alphanumeric characters or underscores". The name was right; the invisible space wasn't. Type these names by hand.

2. The whole name-and-value pair got pasted into the **Name** box. The name goes in the top field, the value in the box below it.

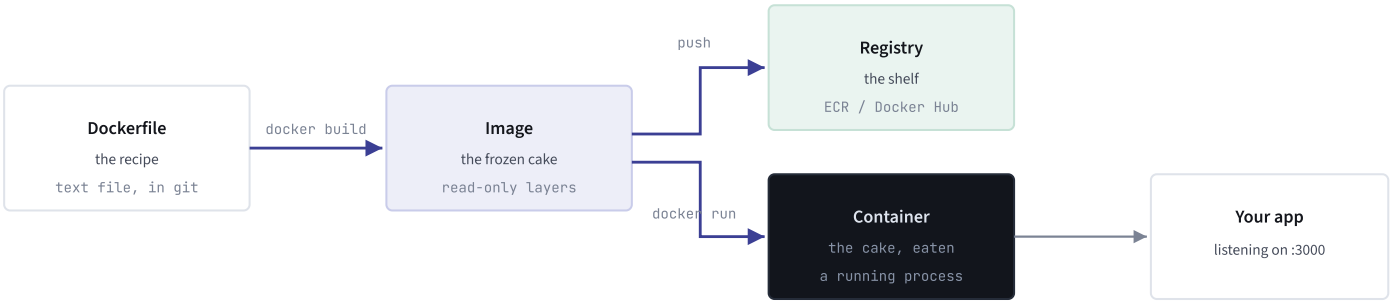
CHECK YOURSELF Â· PART 3

1. You push a new branch called `fix/typo`. Does our CI run? Why or why not?
2. Why should an open pull request *not* push an image to ECR?
3. Why is the IAM role ARN stored as a variable rather than a secret? What would have to be true for it to need to be a secret?

04 Docker: the ideas first

Docker solves one problem: your app runs on your machine and nowhere else. Learn four words and the rest follows.

Image, container, registry



THE THREE NOUNS OF DOCKER, AND THE VERBS BETWEEN THEM

WORD	DEFINITION	ANALOGY
Dockerfile	A text file of instructions for building an image	The recipe
Image	A frozen, read-only snapshot of a filesystem plus a start command	The finished cake in the freezer
Container	A running instance of an image, with its own isolated filesystem and network	Taking the cake out and eating it
Registry	A server that stores images so other machines can download them	The shelf you leave the cake on
Layer	One step of the Dockerfile, cached separately. Unchanged layers are reused.	Why rebuilds are fast

One image, many containers. The image never changes. You can start ten containers from it; each gets its own writable scratch space on top, thrown away when the container is removed. That is why a database inside a container needs a *volume* (Part 7) or its data dies with it.

STEP 4.1 IMPLEMENTED

Install Docker on the EC2 server

PURPOSE

A fresh server has no Docker. Without it there is nothing to run your image.

COMMANDS

UBUNTU Â€” THE QUICK WAY

```
sudo apt update && sudo apt upgrade -y
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
```

UBUNTU Â€” DOCKER'S OFFICIAL APT REPOSITORY (CONTROL OVER VERSIONS)

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
$VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker ubuntu
```

AMAZON LINUX 2023

```
sudo dnf install -y docker
sudo systemctl enable --now docker
sudo usermod -aG docker ec2-user
```

PLAIN ENGLISH

`apt` / `dnf` are package managers â€” they download and install software. `sudo` means "run this as administrator". `systemctl enable --now docker` starts the Docker background service *and* makes it start on every reboot. `usermod -aG docker ubuntu` adds your login to the `docker` group so you can run `docker` without `sudo`.

EXPECTED

After logging out and back in:

```
docker --version
docker compose version
docker run --rm hello-world
```

The last command downloads a tiny test image and prints a success message.

PROBLEM

1. `usermod: group 'docker' does not exist` â€” that group is created *by* the Docker installer. We ran `usermod` before installing Docker. Install first.

2. Permission denied on `/var/run/docker.sock` right after adding the group. A group change does not apply to a shell that is already open â€” run `newgrp docker`, or log out and SSH back in.

3. `apt install docker.io` also works and creates the group, but ships an older Docker **without** the `compose` and `buildx` plugins.

CHECK YOURSELF Â· PART 4

1. Explain the difference between an image and a container to someone who has never used Docker.
2. You add yourself to the `docker` group and immediately get "permission denied". What is wrong, and what are the two ways to fix it?
3. You delete a container and start a new one from the same image. What happened to any files the old container wrote?

05 The Dockerfile & multi-stage build

Our Dockerfile sets up three throwaway environments and ships only the last one.

Why "multi-stage"

Building a Next.js app needs a compiler, all the dev dependencies, and the full source tree. *Running* it needs almost none of that. A multi-stage build lets you use a fat environment to build, then copy just the finished output into a clean, small final image. The build tools never ship.

STAGES 1 AND 2 ARE DISCARDED. ONLY STAGE 3 BECOMES THE IMAGE.

Our Dockerfile, line by line

DOCKERFILE Â€” PROJECT ROOT IMPLEMENTED

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm \
    npm ci \
      --fetch-timeout=600000 \
      --fetch-retries=5 \
      --fetch-retry-mintimeout=20000 \
      --fetch-retry-maxtimeout=120000

FROM node:22-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

INSTRUCTION	WHAT IT DOES
`FROM node:22-alpine AS deps`	Start from the official Node 22 image built on Alpine Linux (tiny, ~60 MB). `AS deps` names the stage so later stages can copy from it.
`WORKDIR /app`	Create `/app` and make it the current directory for everything after.
`COPY package.json package-lock.json ./`	Copy *only* the two dependency files first. Deliberate: Docker caches each step, and this one is invalidated only when dependencies change â€” not every time you edit a component.
`npm ci`	"Clean install." Deletes `node_modules`, installs the exact versions locked in `package-lock.json`, never edits the lockfile, and errors out if the two files disagree. That determinism is why builds use `ci`, not `install`.
`--mount=type=cache,target=/root/.npm`	A BuildKit **cache mount**. Keeps npm's download cache alive between builds without baking it into the image. When the layer is invalidated, npm still finds most package tarballs already on disk.
The four `--fetch-*` flags	Network patience, in milliseconds: allow one request 10 minutes; retry 5 times; wait at least 20s before retrying; cap the backoff at 2 minutes. Defaults are 5 min / 2 / 10s / 60s. Added after a real failure â€” see Part 16.
`COPY --from=deps ...`	Reach into the finished `deps` stage and take its `node_modules`.
`COPY . .`	Copy the whole project into the image â€” filtered by `.dockerignore`.
`RUN npm run build`	Compile the Next.js app. Because of `output: "standalone"`, this produces `.next/standalone`.
`ENV NODE_ENV=production`	Tells Node and Next to run in production mode.
The three `COPY --from=builder` lines	Take only what running needs: the standalone server, the compiled static assets, and `public`. Source code and dev dependencies stay behind.
`EXPOSE 3000`	Documentation: "this container listens on 3000." It does **not** open a port by itself â€” `-p` at run time does that.
`CMD ["node", "server.js"]`	The command run when a container starts. `server.js` is generated by Next's standalone output.

ONE REQUIREMENT TO REMEMBER

`--mount=type=cache` needs **BuildKit**, the modern Docker build engine. It is the default on any current Docker. On an old daemon, or anywhere `DOCKER_BUILDKIT=0` is set, that line is a syntax error. If you hit that, build with `DOCKER_BUILDKIT=1 docker build .`

STEP 5.1 IMPLEMENTED

.dockerignore â€″ keep secrets and junk out of the build

PURPOSE

`COPY . .` copies *everything* in the folder. Without a filter that includes your SSH private key, your `.env`, your `.git` history and a 500 MB `node_modules`.

CONFIG

.DOCKERIGNORE

```
node_modules
.next
.git
.env
.env.*
Dockerfile
.dockerignore
npm-debug.log
README.md
*.pem
nasir-ec2-server.pem
```

PLAIN ENGLISH

Same idea as `.gitignore`, but for Docker. Each line is a pattern that will not be sent to the build. `*.pem` means "any file ending in .pem" â€” that already covers `nasir-ec2-server.pem`, so the last line is redundant but harmless.

PROBLEM

The original file had **no `*.pem` line**, and `nasir-ec2-server.pem` â€” the private SSH key for the server â€” sat in the project root. Line 9 of the Dockerfile does `COPY . .`, so the key was being copied into a build layer. It did not reach the final image, but that was luck (multi-stage), not design. We added the two `.pem` lines.

RULE TO KEEP

An SSH private key should not live in a project folder at all. Its home on Windows is `C:\Users\<you>\.ssh\`. `.gitignore` and `.dockerignore` protect you from git and Docker â€” not from a zip, a backup, or a folder sync.

CHECK YOURSELF Â· PART 5

1. Why does the Dockerfile copy `package.json` and the lockfile *before* copying the rest of the source?
2. What is the difference between `npm install` and `npm ci`, and why does CI want `ci`?
3. The final image contains no source code. How does the app still run?
4. `EXPOSE 3000` is in the Dockerfile, but `http://your-server:3000` doesn't load. Name two separate things that could be missing.

06 Building & tagging the image

STEP 6.1 IMPLEMENTED

Build, tag, push â€″ by hand

PURPOSE

To understand what the pipeline will later do for you. Do it manually first; automation you don't understand is a black box that eventually breaks.

COMMANDS

POWERSHELL, ON YOUR LAPTOP

```
# 1. build, targeting the server's CPU architecture
docker build --platform linux/amd64 -t kanban-app:latest .

# 2. log Docker in to ECR (this token lasts 12 hours)
aws ecr get-login-password --region us-east-1 `
  | docker login --username AWS --password-stdin 239068269281.dkr.ecr.us-east-
1.amazonaws.com

# 3. give the image its full registry address
docker tag kanban-app:latest 239068269281.dkr.ecr.us-east-
1.amazonaws.com/kanban-app:latest

# 4. upload it
docker push 239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app:latest
```

PLAIN ENGLISH

`-t` is **tag** â€” the image's name. The trailing `.` is the **build context**: the folder Docker sends to the builder (filtered by `.dockerignore`). `docker tag` copies nothing; it adds a second name pointing at the same image. A registry works out where an image belongs purely from its name, which is why the full ECR address has to be in the tag before you can push.

EXPECTED

`Login Succeeded`, then a series of layer uploads, then the tag appears in the ECR console.

THE CONSOLE WILL WRITE THESE FOR YOU

ECR â†' Repositories â†' click your repo â†' **View push commands**. AWS prints all four commands with your real account ID already filled in.

Image tags: why we push two

TAG	EXAMPLE	PURPOSE
`$GITHUB_SHA`	`â€¦/kanban-app:7585c80â€¦`	**Immutable identity.** This tag will always mean exactly this commit. It is what makes rollback possible.
`latest`	`â€¦/kanban-app:latest`	**Convenience pointer.** Whatever was pushed most recently. The server pulls this.

These are two names for **one** image â€” ECR stores the layers once and shows one entry with two tags. Rolling back is then simply: run the previous SHA tag instead of `latest`.

ARCHITECTURE MISMATCH Â€" THE MISTAKE THAT COSTS AN AFTERNOON

An image built for Intel/AMD chips will not run on ARM, and vice versa. It fails at start-up with `exec format error`.

- GitHub Actions runners are `x86_64` â†' `linux/amd64`
- `t2.*`, `t3.*`, `m5.*` EC2 instances are also `linux/amd64` â†' match, nothing to do
- `t4g.*`, `r8g.*` (AWS Graviton) are ARM â†' you must build `linux/arm64`

Check on the server with `uname -m`: `x86_64` â†' amd64, `aarch64` â†' arm64. Our instance is x86, so we needed nothing special.

CHECK YOURSELF Â· PART 6

1. Why must the image be tagged with the ECR address *before* pushing, rather than telling `docker push` where to send it?
2. Both tags point at the same image. What does the SHA tag give you that `latest` cannot?
3. You launch a `t4g.small` instance and your container immediately exits with `exec format error`. What happened?

07 Container basics

The run command, dissected

```
docker run -d --name kanban --restart unless-stopped \
  -p 80:3000 \
  --env-file .env \
  239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app:latest
```

FLAG	MEANING
`-d`	Detached â€” run in the background and give you your terminal back.
`--name kanban`	A readable handle, so you can type `docker logs kanban` instead of pasting an ID.
`--restart unless-stopped`	Docker restarts the container if it crashes, and again after a server reboot â€” unless you stopped it deliberately. This is what makes a deploy survive a reboot.
`-p 80:3000`	**host port : container port.** Traffic arriving at the server's port 80 is forwarded to port 3000 inside the container. This is the only reason the outside world can reach the app.
`--env-file .env`	Read `KEY=value` lines from a file *on the server* and set them inside the container â€” at run time, never baked into the image.
the image address	Which image to start. If it isn't on disk, Docker pulls it.

IF THE APP STARTS BUT THE PORT GIVES NOTHING

Add `-e HOSTNAME=0.0.0.0`. Next.js standalone sometimes binds to `localhost` *inside* the container, and port mapping cannot reach a loopback-only listener. `0.0.0.0` means "accept connections on every interface".

Everyday container commands

```
docker ps                    # what is running right now
docker ps -a                 # including stopped containers
docker images                # images on this machine
docker logs -f kanban        # follow the app's output; Ctrl+C stops watching
docker rm -f kanban          # stop and delete the container
docker system prune -a       # delete unused images/layers â€” frees a lot of disk
docker volume ls             # list named volumes
```

`PRUNE -A` IS NOT GENTLE

It deletes every image not currently used by a running container. If your only copy of an image is on that box and it is not running, it is gone. One of the reasons we kept images in ECR rather than deleting them after pulling (Part 10).

Container networking, volumes and ports NOT IN OUR REPO

We drafted a Compose file while experimenting on EC2, but the final pipeline does not use one â€” the deploy is a single `docker run`. Read this section for the concepts; they come up constantly.

```yaml
services:
  mongo:
    image: mongo:8
    restart: always
    volumes:
      - mongo-data:/data/db
    # deliberately NO ports: section

  app:
    build: .
    restart: always
    ports:
      - "3000:3000"
    environment:
      - MONGODB_URI=mongodb://mongo:27017/kanban
    depends_on:
      - mongo

volumes:
  mongo-data:
```

1. A service name is a hostname

Compose creates a private network for the project and runs a small DNS server inside it. From the `app` container, the name `mongo` resolves to the Mongo container's current IP â€” which changes on every restart, and DNS follows it automatically. Two wrong answers, failing for different reasons:

- `localhost` inside a container means *that container* â€” not the host, not a sibling. You get `ECONNREFUSED`. This is the single most common Docker networking mistake.
- An **Atlas URL** points at MongoDB's cloud service â€” correct when you are *not* running your own Mongo, wrong the moment you add a `mongo` service.

Note it is the *service* name, not the container name. Adding `container_name: my-mongo` changes nothing; the service is still reachable as `mongo`.

2. A named volume is where data actually survives

A container's filesystem is a disposable layer. Mongo writes to `/data/db`; without a volume those writes vanish when the container is deleted. `mongo-data:/data/db` mounts a Docker-managed directory on the host (under `/var/lib/docker/volumes/`) over that path, so writes land on the host disk and outlive the container.

> THE ONE FLAG TO FEAR
>
> `docker compose down` stops and removes containers â€” your data is safe.
> `docker compose down -v` **also deletes the named volumes.** That `-v` is the difference between "restart my stack" and "erase my database".

3. No `ports:` means not on the internet

`ports:` publishes a container port onto the *host's* network interface. It is **not** what lets containers talk to each other â€” they already can, on every port, over the private network. So `app` can reach `mongo:27017` all day while nothing outside the server can.

Publishing `27017:27017` would put a database with **no authentication by default** onto a public IP. Automated scanners find open MongoDB instances within hours, and there is a long history of them being wiped and held for ransom. For local debugging only, bind to loopback: `"127.0.0.1:27017:27017"`.

THE IDEA THAT TIES IT TOGETHER

The container network is the trust boundary. Everything inside talks freely by service name; `ports:` is the one deliberate hole you punch, for the one service that genuinely needs public traffic. Here, that is the web app and only the web app.

CHECK YOURSELF Â· PART 7

1. In `-p 80:3000`, which number is the server's port and which is the container's? What happens if you swap them?
2. Why must a database container have a volume, and what does `docker compose down -v` do that `down` does not?
3. Your app cannot reach the database at `mongodb://localhost:27017`. Why not, and what should the hostname be?
4. What does `--restart unless-stopped` protect you from?

08 GitHub Actions: our CI pipeline

One YAML file in your repo. GitHub reads it, rents a fresh Linux machine, and runs your steps on it. No server for you to maintain.

The vocabulary

TERM	MEANING
Workflow	One YAML file in `.github/workflows/`. Ours is `ci.yml`.
Trigger (`on:`)	The events that start it. Ours: push to `main`, or a PR targeting `main`.
Job	A group of steps that run on one machine. Ours is called `build`.
Runner (`runs-on:`)	The machine GitHub rents for you. `ubuntu-latest` â€” clean, and destroyed afterwards.
Step	One thing to do. Either `uses:` (someone else's prebuilt action) or `run:` (a shell command).
Action	A reusable package, referenced as `owner/name@version`, e.g. `actions/checkout@v4`.
Permissions	What the run is allowed to do. Important for OIDC â€” see Part 12.

STEP 8.1 IMPLEMENTED

Where we started: checks only

WHAT WE DID

The first version of `ci.yml` installed dependencies, checked formatting, linted, and built the Docker image purely as a validation step. The image was never tagged, never pushed, and disappeared when the runner shut down.

CONFIG

.GITHUB/WORKFLOWS/CI.YML Â€” THE ORIGINAL

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Check formatting
        run: npm run format:check

      - name: Lint
        run: npm run lint

      - name: Build Docker image
        run: docker build -t kanban-app .
```

PLAIN ENGLISH

`actions/checkout@v4` â€” downloads your repository onto the runner. Without it the machine is empty.
`actions/setup-node@v4` with `cache: npm` â€” installs Node 22 and caches the npm download folder between runs, so `npm ci` is faster next time.
`npm run format:check` â€” runs `prettier --check .`: "is every file formatted correctly?" It changes nothing; it just fails if not.
`npm run lint` â€” runs ESLint, which finds likely bugs and style problems.
`docker build -t kanban-app .` â€” proves the Dockerfile still works.

EXPECTED

A green tick on every PR and every commit to `main`. A red cross if formatting, linting, or the Docker build fails â€” and a red cross on a PR blocks the merge.

> LIMITATION
>
> This is CI with no CD. The image is built and thrown away. Nothing leaves the runner, so nothing can be deployed. Closing that gap is the whole of Parts 10â€“13.

Where we ended: build, then publish

The finished workflow does the same checks, then â€” **only on a push to `main`** â€” authenticates to AWS, logs in to ECR, and pushes the image under two tags.

.GITHUB/WORKFLOWS/CI.YML Â€” CURRENT `IMPLEMENTED`

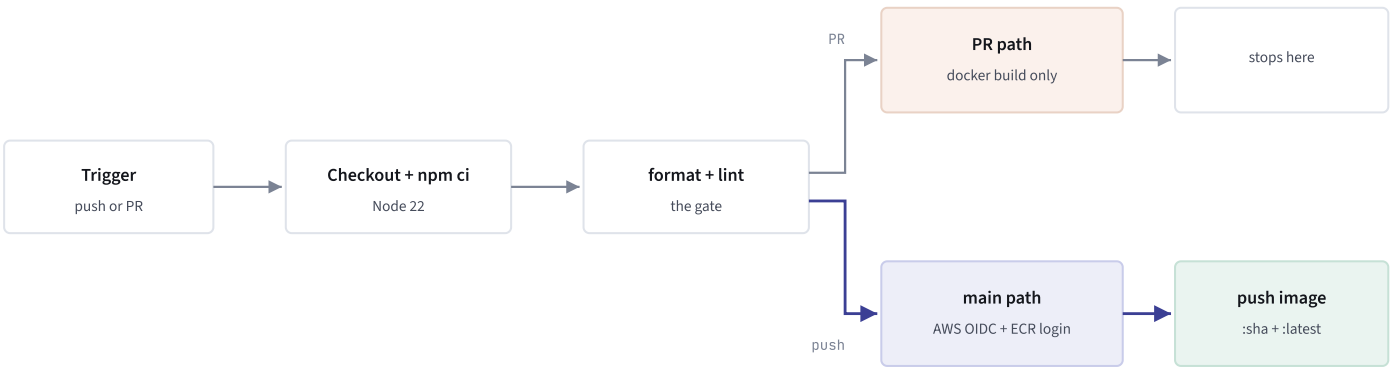
```yaml
name: CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    permissions:
      id-token: write        # lets GitHub mint the OIDC token
      contents: read         # lets the checkout step read the repo

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Check formatting
        run: npm run format:check

      - name: Lint
        run: npm run lint

      - name: Build Docker image (PR check)
        if: github.event_name == 'pull_request'
        run: docker build -t kanban-app .

      # (a commented-out OIDC debug step lives here â€” see Part 17)

      - name: Configure AWS credentials
        if: github.event_name == 'push'
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ vars.AWS_ROLE_ARN }}
          aws-region: ${{ vars.AWS_REGION }}
          role-skip-session-tagging: true

      - name: Login to ECR
        id: ecr
        if: github.event_name == 'push'
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build and push
        if: github.event_name == 'push'
        env:
```

```
          REGISTRY: ${{ steps.ecr.outputs.registry }}
          REPO: ${{ vars.ECR_REPOSITORY }}
        run: |
          docker build -t $REGISTRY/$REPO:$GITHUB_SHA -t $REGISTRY/$REPO:latest .
          docker push $REGISTRY/$REPO:$GITHUB_SHA
          docker push $REGISTRY/$REPO:latest
```

The new pieces explained

LINE	WHAT IT DOES
`permissions: id-token: write`	Allows this job to request a short-lived identity token from GitHub. **Miss this and you get the confusing error "Credentials could not be loaded."** It does not grant write access to anything in your repo.
`if: github.event_name == 'push'`	"Only run this step when the trigger was a push." A PR run skips it. Applied to all three AWS steps.
`if: github.event_name == 'pull_request'`	The mirror image, on the PR-only build step.
`aws-actions/configure-aws-credentials@v4`	Trades GitHub's identity token for temporary AWS credentials by assuming your IAM role. Everything after this step in the job is authenticated as that role.
`role-skip-session-tagging: true`	Stops the action attaching session tags to the role assumption. Session tags need extra permission in the trust policy; we removed the variable while debugging (Part 16) and kept it off.
`aws-actions/amazon-ecr-login@v2`	Runs `docker login` against your ECR registry using those temporary credentials.
`id: ecr` â†' `steps.ecr.outputs.registry`	Naming a step lets later steps read its outputs. The ECR login action outputs your registry hostname, so you never hard-code the account ID in the workflow.
`$GITHUB_SHA`	A built-in variable: the full commit hash of the code being built. Used as the immutable image tag.
`run: |`	The pipe means "everything indented below is a multi-line shell script."

What runs when



ONE WORKFLOW, TWO BEHAVIOURS, DECIDED BY THE `IF:` CONDITIONS

STEP	ON A PR	ON PUSH TO MAIN
format:check + lint	runs	runs
`docker build`	runs (PR check step)	runs (inside build-and-push)
AWS credentials, ECR login, push	**skipped**	runs

WHY THE SEPARATE PR BUILD STEP EXISTS

The original advice was "gate the three AWS steps and PRs will still build the image." That was wrong, and we caught it: the `docker build` command lives *inside* the "Build and push" step, so gating that step removes Docker from PRs entirely. Adding a small PR-only build step restores the "does this even build?" signal â€” which is the most useful thing a PR check can tell you.

A word on the alternative: Jenkins EXPLAINED, NOT USED

Jenkins is the older, self-hosted equivalent. The shape of a pipeline is nearly identical; what differs is who runs the server.

GITHUB ACTIONS	JENKINS
Managed by GitHub, no server	A Java app *you* install, patch and back up
`ci.yml` (YAML)	`Jenkinsfile` (Groovy)
`jobs:` / `runs-on:` / `steps:`	`stages { }` / `agent { }` / `steps { sh 'â€¦' }`
`if:`	`when { }`
`${{ secrets.FOO }}`	`credentials('foo')`
`uses: aws-actions/â€¦`	a plugin plus `withAWS() { }`
Ties you to GitHub	Works with any Git host

Jenkins still wins for air-gapped or heavily regulated environments, exotic build hardware, or an organisation that already runs it. For a Next.js app going to ECR and EC2, none of those apply â€” Jenkins would mean maintaining a second server to do what GitHub already does for free. Worth an afternoon of hands-on time for interviews; not worth adopting here.

CHECK YOURSELF Â· PART 8

1. What does `actions/checkout@v4` do, and what would happen if you deleted that step?
2. Explain `permissions: id-token: write` in one sentence. What is the error message when it is missing?
3. Why does the workflow have *two* `docker build` commands? Do both ever run in the same job?
4. Where does `steps.ecr.outputs.registry` get its value, and why is that better than typing the account ID into the YAML?

09 AWS EC2 setup

EC2 rents you a computer in a data centre. Everything else in this part is about getting into it and letting the right traffic reach it.

The vocabulary

TERM	MEANING
EC2	Elastic Compute Cloud â€” virtual machines you rent by the second.
Instance	One running virtual machine.
Instance type	The *hardware shape* only: vCPUs, RAM, network. `t3.micro` = 2 vCPU / 1 GB. It contains no operating system.
AMI	Amazon Machine Image â€” the disk image: Ubuntu, Amazon Linux, Windows. **Instance type + AMI = a real VM.**
Key pair	The SSH key. AWS keeps the public half; you download the private `.pem` *once*, at creation, and it can never be downloaded again.
Security group	A firewall in front of the instance. Default: block all incoming traffic.
Region	The physical location, e.g. `us-east-1`. Resources are per-region â€” something can hide in a region you aren't looking at.
Elastic IP	A fixed public address. Without one, the public IP changes every stop/start.

READING INSTANCE TYPE NAMES

`r8gb.8xlarge` = family `r` + generation `8` + variant letters `gb` + size `8xlarge`.
Family: `t` burstable/cheap Â· `m` general purpose Â· `c` compute-heavy Â· `r` memory-heavy Â· `i` / `d` storage-heavy.
Suffixes: `g` Graviton/ARM Â· `i` Intel Â· `a` AMD Â· `d` local NVMe SSD Â· `n` / `b` extra network or block bandwidth.
The giant ones in the catalogue (`i8ge.24xlarge` and friends) are $2â€“20 *per hour* machines for data warehouses and ML training. **Browsing that list costs nothing and launches nothing** â€” billing starts only when an instance is actually running. Filter by "Free tier eligible" and ignore the rest permanently.

STEP 9.1 IMPLEMENTED

Launch the instance

WHAT WE DID

EC2 â†’ **Launch instances**, with these choices:

SETTINGS

Name	anything â€” it is just a display tag, not unique, unrelated to your ECR repo name
AMI	Ubuntu Server 24.04 LTS (or Amazon Linux 2023)
Instance type	`t3.micro` â€” free-tier eligible, 1 GB RAM
Key pair	reuse your existing one from the dropdown
Storage	20 GiB gp3 â€” 8 GiB fills up fast with Docker images
Advanced â†’ IAM instance profile	a role with `AmazonEC2ContainerRegistryReadOnly` (can also be attached later)

PROBLEM

We first launched a **`t2.nano`** â€” 0.5 GB RAM. It was the wrong choice twice over: it is *not* free-tier eligible (about $4/month) and it has half the RAM of the `t3.micro` that is free. A Next.js build wants 1â€“2 GB; on 512 MB the kernel's out-of-memory killer stops the build with a bare `Killed` and no explanation.

FIX

Do not terminate â€” resize. Stop the instance (Stop, *not* Terminate) â†’ Actions â†’ Instance settings â†’ Change instance type â†’ `t3.micro` â†’ Start. The disk, the Docker install, and the key pairing all survive. Only the public IP changes.

TYPE	RAM	VERDICT FOR THIS APP
`t2.nano`	0.5 GB	Only workable with a hosted database, off-box builds, and swap
`t3.micro`	1 GB	Comfortable for Next.js + hosted Mongo. Still don't build on it. **Free tier.**
`t3.small`	2 GB	Can self-host MongoDB *and* build on the box

STEP 9.2 IMPLEMENTED

Open the right ports (security group)

PURPOSE

AWS blocks **all** incoming traffic by default. Your app can be running perfectly and still time out for every visitor. This rule is what makes the door exist.

WHAT WE DID

EC2 â†’ your instance â†’ **Security** tab â†’ click the security group â†’ **Edit inbound rules**:

RULES

TYPE	PORT	SOURCE	WHY
SSH	22	**My IP**	Only you should be able to log into the server
HTTP	80	0.0.0.0/0	Anyone must be able to load your site
HTTPS	443	0.0.0.0/0	Needed later, once you add a domain and a certificate
Custom TCP	3000	0.0.0.0/0	Only if you run the container as `-p 3000:3000` instead of `-p 80:3000`

Never open 27017 (MongoDB) to the internet.

PLAIN ENGLISH

The **source** answers "who is allowed to connect to this port". `0.0.0.0/0` means the whole internet. **My IP** auto-fills your current address as `x.x.x.x/32` â€” the `/32` means exactly one address. Each port has its own independent source, so setting 80 to Anywhere does not touch the SSH rule.

EXPECTED

Rules take effect **immediately** â€” no restart, no downtime. A saved rule shows a `sgr-â€¦` ID in the first column.

PROBLEM

1. We added the port-3000 row but left the **Source blank** and never pressed **Save rules**. The giveaway: the rule's ID column showed `â€“` instead of `sgr-â€¦`. A half-entered rule does nothing.
2. If your home ISP rotates your IP, a "My IP" SSH rule silently stops working and SSH just *hangs*. That is the first thing to re-check when a connection times out.

STEP 9.3 IMPLEMENTED

Connect over SSH from Windows

PURPOSE

The browser-based "EC2 Instance Connect" cannot copy files to the server, which makes everything harder. With the `.pem` you can use PowerShell's built-in OpenSSH.

COMMANDS

```
# 1. lock the key file down (Windows only)
icacls "d:\projects\Kanban-App\kanban-app\nasir-ec2-server.pem" /inheritance:r
icacls "d:\projects\Kanban-App\kanban-app\nasir-ec2-server.pem" /grant:r
"$($env:USERNAME):(R)"

# 2. connect
ssh -i "d:\projects\Kanban-App\kanban-app\nasir-ec2-server.pem"
ubuntu@3.87.232.6
```

PLAIN ENGLISH

`-i` points at your private key file.
The username is decided by the AMI: **Ubuntu â†' `ubuntu`**, **Amazon Linux â†' `ec2-user`**, Debian â†' `admin`. Getting it wrong gives `Permission denied (publickey)`.
`icacls /inheritance:r` strips the permissions the file inherited from its folder; `/grant:r "user:(R)"` then gives read access to you alone. OpenSSH refuses to use a key other accounts can read.

EXPECTED

First connection asks you to trust the host fingerprint â€" type `yes`. Then a shell prompt like `ubuntu@ip-172-31-81-199:~$`.

PROBLEM

`UNPROTECTED PRIVATE KEY FILE` â€" the permissions fix above. Note that `icacls` would **not** apply `/inheritance:r` and `/grant` in one command; the inherited entries survived. It took two separate passes, plus an explicit removal:

```
icacls "â€¦\nasir-ec2-server.pem" /inheritance:r
icacls "â€¦\nasir-ec2-server.pem" /grant:r "NasirPC:(R)"
icacls "â€¦\nasir-ec2-server.pem" /remove "NT AUTHORITY\Authenticated Users"
"BUILTIN\Users" "BUILTIN\Administrators" "NT AUTHORITY\SYSTEM"
```

The result should be exactly one entry: your own user, with `(R)`.

SYMPTOM	MEANING
Hangs, then times out	Security group is not allowing your IP on port 22 â€" or your IP changed
`Permission denied (publickey)`	Wrong username for the AMI, or the wrong key file
`UNPROTECTED PRIVATE KEY FILE`	File permissions â€" run the `icacls` commands
`Connection closed by â€¦ port 22`	Different animal: SSH *answered* then dropped you. See below.

"CONNECTION CLOSED" Â€” OUR WORST OUTAGE

Both PowerShell SSH *and* the browser Instance Connect failed the same way. That combination proved the problem was on the instance, not our key or firewall: sshd answers the TCP handshake but cannot give you a session. On a `t2.nano` with no swap the two overwhelming causes are:

- OUT OF MEMORY
  â€” a build OOM-killed things and left sshd unable to fork a session
- DISK FULL
  â€” Docker images filled the 8 GB root volume; sshd cannot write a session file

Recovery order: (1) Reboot from the console, wait for `2/2 checks passed`, retry. (2) If still failing, Actions â†’ Monitor and troubleshoot â†’ **Get system log**, and look for `Out of memory: Killed process` or `No space left on device`. (3) Once back in: `df -h`, `free -m`, `docker system prune -a`.
The durable fix is not to build on a tiny box at all â€” which is exactly what the CI pipeline gives you.

STEP 9.4 EXPLAINED

Add swap, if you must build on the server

PURPOSE

Swap is disk space the kernel uses as overflow when RAM runs out. It is slow, but it turns a hard crash into a slow build.

COMMANDS

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

PLAIN ENGLISH

`fallocate` creates a 2 GB file; `chmod 600` makes it readable only by root; `mkswap` formats it as swap; `swapon` activates it. The last line adds it to `/etc/fstab` so it is re-enabled after a reboot.

EXPECTED

`free -m` now shows a Swap row of about 2048.

STEP 9.5 IMPLEMENTED

Terminating and starting over

WHAT WE DID

We terminated the `t2.nano` and launched a clean `t3.micro`, reusing the same key pair.

CHOICES

ACTION	COMPUTE CHARGE	STORAGE CHARGE	YOUR DATA
Stop	stops	continues (~$0.08/GB/month)	kept
Terminate	stops	stops (root volume deleted)	**gone**
Wipe and reuse	continues	continues	your choice

PLAIN ENGLISH

EC2 â†’ Instances â†’ tick the instance â†’ **Instance state** â†’ "Terminate (delete) instance". If it is greyed out, termination protection is on (Actions â†’ Instance settings â†’ Change termination protection). Billing stops the moment the state reads `terminated`; the row lingers in the console for 30â€“60 minutes and then vanishes on its own.

WATCH FOR

Terminating an instance does **not** stop every charge. Check all four:

CLEANUP

CONSOLE PAGE	LOOK FOR
EC2 â†’ Volumes	Volumes in state `available` â€” detached but still billed
EC2 â†’ Elastic IPs	An IP not attached to a running instance â€” about $3.60/month
EC2 â†’ Snapshots	Old backups
Billing â†’ Bills	The definitive view â€” it covers **all regions at once**

SET A BUDGET ALERT BEFORE YOU EXPERIMENT FURTHER

Billing â†’ Budgets â†’ Create budget â†’ **Zero spend budget** â†’ your email. It takes a minute and emails you the moment anything starts charging. If billing pages show "Access Denied", you are signed in as an IAM user: sign in as root â†’ Account settings â†’ **IAM user and role access to Billing Information** â†’ Activate, then attach the `Billing` policy to the IAM user. Both steps are needed â€” one allows it, the other grants it.

CHECK YOURSELF Â· PART 9

1. What is the difference between an instance type and an AMI? Which one decides your SSH username?
2. Your SSH connection hangs with no error. What is the first thing to check â€” and how does that differ from what you'd check for `Permission denied (publickey)`?
3. Why is `t3.micro` a better choice than `t2.nano` on both cost *and* capability?
4. You terminate your instance. Name two things that can keep charging you afterwards.

10 Amazon ECR: storing the image

A registry is the shelf between "built" and "running". Without one, the image only exists on the machine that built it â€” and CI runners are destroyed after every run.

Why a registry at all

Before we had ECR, the workflow's `docker build` produced an image on a rented Ubuntu machine that GitHub deletes minutes later. There was no way for the EC2 server to ever see it. A registry is the shared, durable place both sides can reach.

	AMAZON ECR	DOCKER HUB
Private by default	Yes	No â€” free accounts make images **public**
Credentials on the server	None â€” an IAM role does it	A username and access token you must store
Pull cost from same-region EC2	Free	Normal internet transfer
Storage	$0.10 / GB-month	Free tier, then paid
Verdict for us	**Chosen** â€” already in AWS, private, no keys on the box	Fine, but an extra account and extra secrets

STEP 10.1 IMPLEMENTED

Create the repository

WHAT WE DID

Console â† search **ECR** â† Elastic Container Registry â† check the region reads **N. Virginia (us-east-1)** â† **Repositories** â† **Create repository** â† Private â† name `kanban-app` â† everything else default â† Create.

CLI EQUIVALENT

```
aws ecr create-repository --repository-name kanban-app --region us-east-1
```

EXPECTED

The repository appears with a **URI** â€” and that one string contains three of the values you will need repeatedly:

```
239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app
â””â”€â”€â”€ account ID â”€â”˜        â”” region â”˜          â”” repo name â”˜
```

PROBLEM

The newer console has no visible "Visibility" radio button â€” it moved to the left sidebar as separate **Private registry** and **Public registry** sections. If you are under "Private registry â† Repositories", the choice is already made.
Also: the repository name has nothing to do with your instance name. It is the image's name and appears in every image address.

KEEP THE REGION CONSISTENT

The ECR repository must sit in the **same region as the EC2 instance** that pulls from it. Same region = free data transfer; different region = you pay on every pull, forever.

What ECR actually costs EXPLAINED

- **Creating a repository is free.** An empty repo is $0 â€” no per-repo fee, no minimum. Nothing appears in billing until you push.
- **Storage: $0.10 per GB-month** for private repos, prorated.
- **Data in is free. Data out to EC2 in the same region is free.**
- The 12-month free tier includes 500 MB-month of private storage. AWS reworked free-tier terms in 2025 for new accounts, so check the Free Tier page under Billing.
- The size ECR bills is the **compressed** layer size â€” typically 30â€“50% of what `docker images` shows.

Practical maths: a ~100 MB Next.js image bills as perhaps 40â€“60 MB. Keeping three tags is roughly $0.03/month. **Compared to the EC2 instance, ECR is a rounding error.**

WE CONSIDERED DELETING IMAGES AFTER PULLING. DON'T.

It technically works â€” once pulled, the layers live in the instance's `/var/lib/docker` and the container keeps running. But it saves pennies while removing: your only rollback, your ability to launch a second server, and your ability to recover from a `docker system prune -a`. The right cost control is a **lifecycle policy**, not deletion.

STEP 10.2 DISCUSSED, NOT APPLIED

Lifecycle policy, so storage never creeps

PURPOSE

Every CI run adds a new SHA tag. Over months that quietly grows. A lifecycle policy expires old images automatically.

CONFIG

```
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Expire untagged images after 1 day",
      "selection": { "tagStatus": "untagged", "countType": "sinceImagePushed",
                     "countUnit": "days", "countNumber": 1 },
      "action": { "type": "expire" }
    },
    {
      "rulePriority": 2,
      "description": "Keep only the last 3 tagged images",
      "selection": { "tagStatus": "any", "countType": "imageCountMoreThan",
                     "countNumber": 3 },
      "action": { "type": "expire" }
    }
  ]
}
```

```
aws ecr put-lifecycle-policy --repository-name kanban-app \
  --lifecycle-policy-text file://ecr-lifecycle.json
```

EXPECTED

Storage stays capped at roughly 1 GB forever, while always keeping a working image to redeploy or roll back to.

Inspecting what you have stored

```
aws ecr describe-images --repository-name kanban-app \
  --query "imageDetails[].{tag:imageTags[0],sizeMB:imageSizeInBytes,pushed:imagePushedAt}"
```

Shared layers between tags are not double-counted, so two tags of the same image cost what one costs.

CHECK YOURSELF Â· PART 10

1. Why can't the EC2 server just use the image that GitHub Actions built?
2. Read this apart: `239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app:latest`. Name all four components.
3. Why does ECR need no credentials stored on the EC2 instance, while Docker Hub would?
4. What is the real cost of deleting an image from ECR after pulling it â€” beyond the pennies saved?

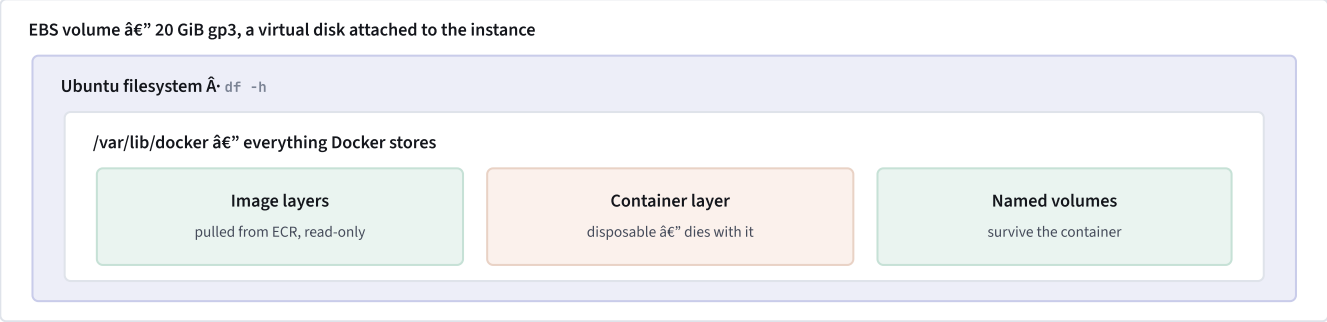
11 EBS & storage concepts

TWO AWS SERVICES, CONFUSINGLY SIMILAR NAMES

EBS = Elastic Block Store â€” virtual hard disks for EC2 instances. **This is what we used**, without ever visiting its console page: the instance's root volume is an EBS volume.

Elastic Beanstalk â€” an entirely different service that deploys apps for you. **We never used it.** If a tutorial says "EBS", check which one it means.

How storage is layered in our setup



EVERYTHING DOCKER DOES LANDS ON ONE EBS VOLUME Â€” WHICH IS WHY IT FILLS UP

What you actually need to know

FACT	CONSEQUENCE
The root volume is EBS, billed per GB-month (~$0.08/GB)	A **stopped** instance still bills for its disk. "Stopped" is not "free".
A terminated instance usually deletes its root volume	â€¦but a volume left in state `available` keeps billing forever. Always check EC2 â†’ Volumes after terminating.
8 GiB is the default root size	Docker base image + OS uses ~3 GB. Build caches and layers can eat another 2â€“3 GB. We recommend **20 GiB gp3**.
A full disk breaks SSH	`sshd` cannot write a session file, so it closes your connection with no useful message. See Part 9's outage.
Docker volumes live on the same disk	Under `/var/lib/docker/volumes/<name>/_data`. Database data is on the EBS volume, not "inside Docker" as some separate place.
Volume names get a project prefix	`mongo-data` declared in a project folder called `kanban-app` becomes `kanban-app_mongo-data`. That is how two projects can both use the name.

Disk hygiene commands

```
df -h                              # how full is the disk
free -m                            # how much RAM and swap is left
docker system df                   # what Docker specifically is using
docker system prune -a             # reclaim unused images and build cache
docker volume inspect kanban-app_mongo-data   # where a volume lives on disk
```

CHECK YOURSELF Â· PART 11

1. What is the difference between EBS and Elastic Beanstalk, and which one did we use?
2. You stop an instance overnight to save money. What are you still paying for?
3. Why can a full disk cause "Connection closed by â€¦ port 22"?
4. Where does a Docker named volume physically live?

12 Connecting CI to AWS with OIDC

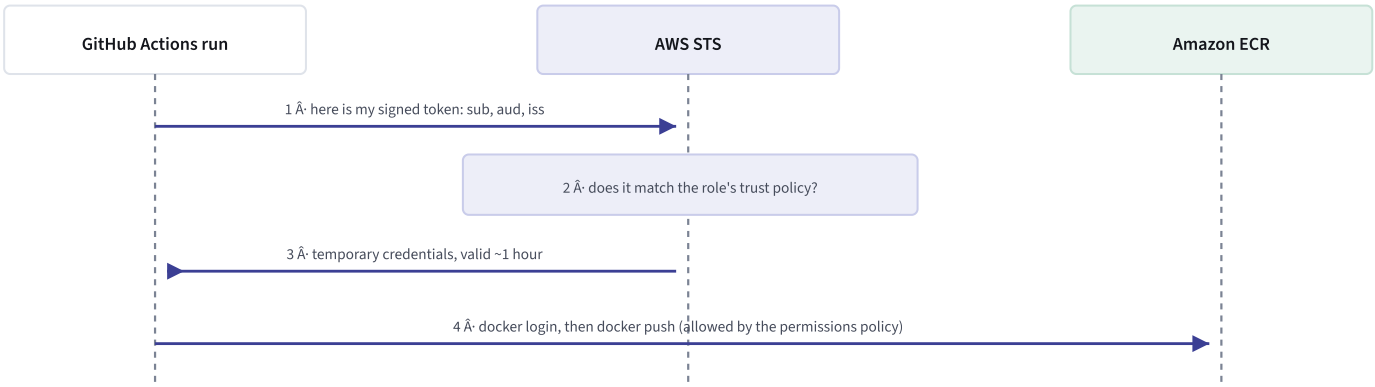
The hardest part of our whole project, and the part most worth understanding. GitHub's servers are not yours, but they need to prove to AWS that they are allowed to push.

Two ways to prove identity

	ACCESS KEYS	OIDC
What it is	A permanent ID + secret pair you paste into GitHub Secrets	A short-lived signed token GitHub mints for each run
Analogy	Copying your house key and mailing it to GitHub	GitHub shows ID at the front desk and gets a badge that expires after the meeting
Stored in GitHub	Yes, forever	Nothing
If it leaks	Attacker has your AWS access until you notice and revoke	Already expired, useless
Rotation	You must remember to	Automatic â€” credentials last about an hour
Setup effort	5 minutes	~20 minutes, once

We chose OIDC â€” *OpenID Connect*, an open standard for one system to prove an identity to another. Nothing secret is ever stored in the repository.

How the handshake works



NO PASSWORD EXISTS ANYWHERE IN THIS DIAGRAM

The five words you must not confuse

WORD	VALUE IN OUR SETUP	WHAT IT MEANS
Issuer (`iss`) â€” AWS calls the field "Provider URL"	`https://token.actions.githubusercontent.com`	Who signs the ID cards. Identical for every GitHub.com user; you register it once per AWS account.
Audience (`aud`)	`sts.amazonaws.com`	Who the card is addressed to. Stops a card meant for AWS being replayed against Azure. It is the hostname of AWS's Security Token Service.
Subject (`sub`)	`repo:Nasir1504@87933341/kanban-board@1345866624:ref:refs/heads/main`	What is printed on the card: the specific repo and branch. **This is the security boundary.**
Trust policy	on the role	*Who* may become this role
Permissions policy	on the role	*What* they may do once they are it

THE UNIFORM ANALOGY

A role is a uniform hanging in a locker. The **trust policy** is the lock â€” it decides who may put the uniform on. The **permissions policy** is the badge on the uniform â€” it decides which doors that person can then open. Ours says: "any GitHub Actions run from repo X may wear this" and "while wearing it, you may push images to ECR."

STEP 12.1 IMPLEMENTED

Register GitHub as a trusted identity provider

PURPOSE

Tell AWS, once per account: "tokens signed by GitHub are worth checking." Like registering a passport authority.

WHAT WE DID

IAM â†' **Identity providers** â†' **Add provider** â†' **OpenID Connect**

- Provider URL: `https://token.actions.githubusercontent.com`
- Audience: `sts.amazonaws.com`

PLAIN ENGLISH

This URL is the same for every GitHub.com repository on earth â€" there is only one GitHub. It is not a per-account value and there is nothing to look up. You can prove it is real yourself, because OIDC requires every issuer to publish its configuration:

```
curl -s https://token.actions.githubusercontent.com/.well-known/openid-configuration
```

If that returns JSON containing `"issuer": "https://token.actions.githubusercontent.com"`, the URL is confirmed by the provider itself â€" better evidence than any tutorial.

PROBLEM

1. Duplicated scheme. AWS pre-fills `https://` in the box. Pasting the full URL on top produced `https://https://token.actions.githubusercontent.com`. Clear the field completely first.
2. No "Get thumbprint" button. Older guides say to click it. AWS removed that step â€" it now validates the host against its own trusted certificate authorities, the way a browser does. Nothing is missing; the guide is out of date. (Scripted setups via CLI or Terraform may still *require* a thumbprint argument, which is why you see the fossil value `6938fd4d98â€¦` copied around â€" it satisfies a required parameter and provides no security.)
3. Already exists. One provider per AWS account. A second attempt fails with `EntityAlreadyExists` â€" reuse the existing one.

EXPECTED

`token.actions.githubusercontent.com` now appears in the Identity providers list, with ARN `arn:aws:iam::239068269281:oidc-provider/token.actions.githubusercontent.com`.

STEP 12.2 IMPLEMENTED

Create the role GitHub will assume

WHAT WE DID

IAM â†’ **Roles** â†’ **Create role** â†’ **Web identity**. Identity provider: the one just created. Audience: `sts.amazonaws.com`. GitHub organisation `Nasir1504`, repository `kanban-board`. Named it `github-actions-ecr-push`.

TRUST POLICY

ROLE Â› TRUST RELATIONSHIPS Â› EDIT TRUST POLICY

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::239068269281:oidc-
provider/token.actions.githubusercontent.com"
      },
      "Action": [
        "sts:AssumeRoleWithWebIdentity",
        "sts:TagSession"
      ],
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub":
"repo:Nasir1504@87933341/kanban-board@1345866624:*"
        }
      }
    }
  ]
}
```

PLAIN ENGLISH

`Principal.Federated` â€” the ID card must be signed by the GitHub provider we registered.
`sts:AssumeRoleWithWebIdentity` â€” the action of trading a token for credentials.
`sts:TagSession` â€” permission for the caller to attach session tags. Not always needed, but harmless to include.
`aud` must equal `sts.amazonaws.com` exactly.
`sub` is **the** security condition. Without it, every repository on GitHub could assume your role. The `*` at the end matches any branch, tag or context.

TIGHTER VARIANT

Once it works, you can restrict to just your main branch by ending the `sub` with `:ref:refs/heads/main` instead of `:*`. Be aware the `sub` changes shape depending on how the job runs:

SITUATION	`sub` VALUE
Push to main	`repo:OWNER/REPO:ref:refs/heads/main`
Tag push	`repo:OWNER/REPO:ref:refs/tags/v1.0`
Pull request	`repo:OWNER/REPO:pull_request`
Job declares `environment:`	`repo:OWNER/REPO:environment:production` â€” the branch **disappears entirely**

That last row catches people out: a branch-scoped condition will never match a job that uses an `environment:`.

PROBLEM Â€” THE BIG ONE

Our `sub` initially read `repo:Nasir1504/kanban-board:*`, which looked perfectly correct and failed every time. The full story is in Part 16; the short version is that this repository has GitHub's **immutable identifiers** enabled, so the real subject carries numeric IDs: `Nasir1504@87933341` and `kanban-board@1345866624`. Names never matched IDs.

STEP 12.3 IMPLEMENTED

Give the role permission to push

PURPOSE

The trust policy lets GitHub *become* the role. This policy decides what the role may *do*.

WHAT WE DID

Attached the AWS-managed policy `AmazonEC2ContainerRegistryPowerUser` to get the pipeline working, having written out the least-privilege inline policy as the eventual target.

THE THREE ECR MANAGED POLICIES

POLICY	CAN DO	VERDICT
ReadOnly	Pull images only	Right for the EC2 instance, wrong for CI
PowerUser	Pull **and push**	What we used. Downside: it can push to *every* ECR repo in the account.
FullAccess	Push, pull, **plus create and delete repositories** and change permissions	Never for CI. If it leaks, someone can delete your repositories.

Watch out for the near-identical `AmazonElasticContainerRegistryPublic*` policies â€” those are for ECR Public, not what you want.

LEAST-PRIVILEGE VERSION

ROLE Â†’ ADD PERMISSIONS Â†’ CREATE INLINE POLICY Â†’ JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "GetAuthToken",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "PushToKanbanRepo",
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:PutImage"
      ],
      "Resource": "arn:aws:ecr:us-east-1:239068269281:repository/kanban-app"
    }
  ]
}
```

PLAIN ENGLISH

Statement 1 â€” `ecr:GetAuthorizationToken` is what `aws ecr get-login-password` calls. It **must** use `"Resource": "*"`; it is an account-level call and AWS rejects it scoped to a repository ARN. That is a quirk, not a mistake.
Statement 2 â€” the actual upload, which Docker performs in pieces:

- `BatchCheckLayerAvailability` â€" "do you already have this layer?" (skips re-uploading)
- `InitiateLayerUpload` â†' `UploadLayerPart` â†' `CompleteLayerUpload` â€" send one layer in chunks
- `PutImage` â€" write the final manifest and tag

These are scoped to one repository ARN, so the role cannot touch anything else. That is **least privilege**: give a job the smallest permission that does its work.

PROBLEM

The JSON editor showed a red âœ— on the `Resource` line. The cause was leaving the placeholders `<REGION>`, `<ACCOUNT_ID>`, `<YOUR_ECR_REPO_NAME>` in place â€" angle brackets are not a valid ARN. All three values come from one place: the ECR repository URI.

STEP 12.4 IMPLEMENTED

Wire it into the workflow

WHAT WE DID

Added three repository **variables** in GitHub (Part 3), the job-level `permissions` block, and the three AWS steps shown in Part 8.

EXPECTED

Push to `main` â†' Actions run goes green â†' ECR console shows **one image with two tags**: the commit SHA and `latest`.

Should immutable identifiers be turned off? EXPLAINED

No â€" keep them on. Names can be reused: delete a repository, and someone else can create one with that name. A name-based trust policy would then happily hand them your AWS role. Numeric IDs are permanent and never reissued, so the policy grants access to *that specific repository*, not to whoever currently holds the name. A bonus: rename the repo tomorrow and CI keeps working, where a name-based policy would silently break.

The one trade-off is readability. Keep a note: `1345866624 â†' kanban-board`, `87933341 â†' Nasir1504`.

CHECK YOURSELF Â· PART 12

1. Explain the difference between a trust policy and a permissions policy using the uniform analogy.
2. What is the `sub` claim, and what happens if you leave it out of the trust policy?
3. Why must `ecr:GetAuthorizationToken` use `"Resource": "*"` when every other action is scoped to one repository?
4. Nothing secret is stored in the repository. So what actually proves to AWS that the run is yours?
5. Your job adds `environment: production`. Your trust policy pins `sub` to `â€¦:ref:refs/heads/main`. What happens, and why?

13 CD: the deployment step

HONEST STATUS

Everything up to here is built and green: a merge to `main` lands an image in ECR automatically. The steps in this part were worked out in detail but the EC2 side had not been completed when these notes were compiled. Treat this as your next session's checklist, not as a description of something already running.

Four things, not one

"Deploy" sounds like one action. It is four, and missing any one of them leaves you with nothing:

#	WHAT	WITHOUT IT
1	An IAM role attached to the **instance**, allowing ECR pull	`docker login` fails â€” no credentials
2	Docker installed on the instance	No `docker` command at all
3	Log in, pull, run	Nothing is running
4	Port 80 open in the security group	The app runs, but nobody can reach it

STEP 13.1 NEXT UP

Give the instance permission to pull

PURPOSE

So the server can authenticate to ECR without any AWS keys stored on it. This is a **second, separate** role from the GitHub one â€” different consumer, different permission.

WHAT TO DO

IAM â†’ Roles â†’ **Create role** â†’ trusted entity **AWS service** â†’ **EC2** â†’ attach `AmazonEC2ContainerRegistryReadOnly` â†’ name it `ec2-ecr-pull`.
Then EC2 console â†’ your instance â†’ **Actions â†’ Security â†’ Modify IAM role** â†’ select it. No restart needed. (It can also be chosen at launch under Advanced details â†’ IAM instance profile.)

PLAIN ENGLISH

An **instance profile** is a role worn by the machine itself. Anything running on that instance â€” including the AWS CLI â€” automatically picks up temporary credentials from the instance metadata service. Same principle as OIDC: no long-lived secret anywhere.

WHY READONLY

The server only needs to *pull*. If the instance were ever compromised, read-only access cannot overwrite or delete your images. Least privilege again.

STEP 13.2 NEXT UP

Pull and run

COMMANDS

ON THE EC2 INSTANCE, OVER SSH

```
# 1. authenticate Docker to ECR (uses the instance role, no keys)
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin 239068269281.dkr.ecr.us-east-
1.amazonaws.com

# 2. download the newest image
docker pull 239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app:latest

# 3. remove the old container if one exists
docker rm -f kanban 2>/dev/null || true

# 4. start the new one
docker run -d --name kanban --restart unless-stopped \
  -p 80:3000 \
  --env-file /home/ubuntu/.env \
  239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app:latest
```

PLAIN ENGLISH

`get-login-password` asks AWS for a temporary registry password and pipes it straight into `docker login` â€” it is never written to a file or shown on screen. That token lasts **12 hours**; when a pull later says "unauthorized", re-run this one line.
`2>/dev/null || true` means "if there is no container called kanban, don't complain, just carry on" â€” so the script works on both the first run and every run after.

EXPECTED

`docker ps` shows one running container, and `http://<your-public-ip>` loads the Kanban board. Use **http**, not https â€” there is no certificate yet.

WATCH FOR

1. If the container starts but the port gives nothing, add `-e HOSTNAME=0.0.0.0`.
2. The `.env` file must exist *on the instance*. It is gitignored and excluded by `.dockerignore`, so neither `git clone` nor the image brings it. Create it there, or `scp` it across.
3. If you use MongoDB Atlas, the EC2 instance's public IP must be added to Atlas â†’ Network Access, or the connection hangs with no error.

STEP 13.3 NEXT UP

Wrap it in a deploy script

PURPOSE

Four commands you retype every time will eventually be four commands you get wrong. Put them in a file first; automate the file second.

COMMANDS

~/DEPLOY.SH ON THE INSTANCE Â€” WRITTEN WITH A HEREDOC SO NOTHING MANGLES IT

```
cat > ~/deploy.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
REGISTRY=239068269281.dkr.ecr.us-east-1.amazonaws.com
REPO=kanban-app
TAG="${1:-latest}"

aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin "$REGISTRY"
docker pull "$REGISTRY/$REPO:$TAG"
docker rm -f kanban 2>/dev/null || true
docker run -d --name kanban --restart unless-stopped \
  -p 80:3000 --env-file /home/ubuntu/.env "$REGISTRY/$REPO:$TAG"
docker image prune -f
EOF
chmod +x ~/deploy.sh
```

PLAIN ENGLISH

`set -euo pipefail` â€” stop at the first error instead of ploughing on.
`${1:-latest}` â€” use the first argument if given, otherwise `latest`. That is your rollback switch: `./deploy.sh 7585c80â€¦` runs a specific commit's image.
A **heredoc** (`<<'EOF'`) writes a file without an editor. The quotes around `'EOF'` stop the shell touching `$` variables. This is safer than pasting into `nano` over SSH, where auto-indent can cascade the indentation and silently corrupt a YAML or script file.

EXPECTED

A redeploy becomes `./deploy.sh`, and a rollback becomes `./deploy.sh <old-sha>`.

Automating the last mile DISCUSSED, NOT IMPLEMENTED

Once the manual deploy works reliably, the pipeline can run it for you. Two routes we discussed:

ROUTE	HOW	TRADE-OFF
AWS Systems Manager (`ssm send-command`)	A CD job in the workflow tells SSM to run `deploy.sh` on the instance	No SSH key in GitHub, no port 22 exposure. Needs the SSM agent and an extra instance permission. **The better option.**
SSH action (`appleboy/ssh-action`)	The workflow SSHes in with your `.pem` stored as a repo secret	Simple, but puts a long-lived private key into GitHub â€” the exact thing OIDC was chosen to avoid.

DO IT BY HAND AT LEAST THREE TIMES FIRST

The tedium is the lesson. Doing it manually is what teaches you what a deploy actually *is*, so automating it later is not cargo-culting. You will also hit the real problems immediately: the container name already exists, old images are eating the disk, the app was down for eight seconds during the swap. Each of those is a real production concern in miniature.

What the current gap costs you

- **No zero-downtime deploy.** There is a gap between `docker rm -f` and `docker run`. Closing it is what load balancers and health checks exist for.
- **No health check.** If the new image crashes on startup, nothing notices. `--restart unless-stopped` will loop it forever.
- **No staging environment.** Every deploy goes straight to the only server there is.
- **No tests in the pipeline.** Format and lint check style, not behaviour. This is the single biggest hole.

CHECK YOURSELF Â· PART 13

1. Why does the EC2 instance need its *own* IAM role rather than reusing `github-actions-ecr-push`?
2. Why `ReadOnly` for the instance and `PowerUser` for CI?
3. Your `docker pull` worked this morning and says "unauthorized" this afternoon. What happened, and what is the one-line fix?
4. Given the deploy script, how would you roll back to the previous release?

14 How the whole pipeline works

One commit, followed all the way through. This is the section to re-read when the pieces stop feeling connected.

The story of one change

1. **You edit a component** on your laptop and commit to a feature branch.
2. **You push the branch.** Nothing runs â€” the workflow only triggers on `main` and on PRs targeting it.
3. **You open a pull request.** GitHub rents a clean `ubuntu-latest` machine, checks out your code, installs Node 22, runs `npm ci`, then `prettier --check`, then ESLint, then `docker build`. The AWS steps are skipped because `github.event_name` is `pull_request`. A green tick means "this is clean and it builds."
4. **You merge.** That merge is a commit on `main`, which is a `push` event, which starts a second run.
5. **Same checks, then the AWS steps fire.** GitHub mints an OIDC token describing this run â€” the repo, the branch. `configure-aws-credentials` presents it to AWS STS.
6. **AWS checks it against the trust policy** on `github-actions-ecr-push`: right issuer, right audience, matching subject. It returns credentials valid for about an hour.
7. `amazon-ecr-login` **runs** `docker login` against your registry using those credentials, and outputs the registry hostname.
8. **The image is built once and tagged twice** â€” with the commit SHA and with `latest` â€” then both tags are pushed. ECR stores the layers once.
9. **The runner is destroyed.** Everything on it is gone; only the image in ECR survives. That is the whole point of the registry.
10. **On the server**, `deploy.sh` authenticates via the instance role, pulls `:latest`, removes the old container and starts a new one mapping port 80 â†’ 3000.
11. **A visitor** hits the public IP on port 80. The security group allows it, Docker forwards it to 3000 inside the container, and Next.js serves the board.

Where every secret and every permission lives

BOUNDARY	WHAT CROSSES IT	WHAT PROTECTS IT
Laptop â†’ GitHub	Your commits	Your GitHub account
GitHub Actions â†’ AWS	A short-lived OIDC token	The trust policy's `sub` condition
Actions â†’ ECR	Image layers	The role's permissions policy (push only, one repo)
EC2 â†’ ECR	Image layers	The instance role (read only)
Internet â†’ EC2	HTTP requests on 80	The security group
You â†’ EC2	An SSH session	Your `.pem` key + the port-22 rule limited to your IP
Container â†’ database	The Mongo connection	`MONGODB_URI`, injected at run time, never in the image

Notice what is absent: no AWS access key exists anywhere in this system. Not in the repo, not in the workflow, not on the server.

CHECK YOURSELF Â· PART 14

1. Walk through what happens between clicking "Merge" and an image existing in ECR. Name at least five steps.
2. The GitHub runner is destroyed after every run. Why doesn't that lose your work?
3. List every place in this system where a long-lived AWS credential could have been stored â€” and explain what we used instead in each case.

15 Environment variables & secrets

The rule that prevents most leaks: secrets are supplied when the container *runs*, never when it is *built*.

Build time vs run time

	BUILD TIME	RUN TIME
When	`docker build`	`docker run`
How to pass	`--build-arg NAME=value` + `ARG` in the Dockerfile	`--env-file .env` or `-e NAME=value`
Ends up in the image?	**Yes** â€” visible in the layer history to anyone who pulls it	No
Use for	Values Next.js must bake into the client bundle: `NEXT_PUBLIC_*`	Everything secret: `MONGODB_URI`, API keys

THE `NEXT_PUBLIC_` TRAP

Any Next.js variable named `NEXT_PUBLIC_SOMETHING` is compiled into the JavaScript sent to the browser. It must be present at **build** time â€” passing it at run time does nothing. And because it ships to the browser, **it is public by definition**: never put a secret behind that prefix, no matter how it is passed.

Where each value lives in our project

VALUE	LIVES IN	REACHES THE APP HOW
`MONGODB_URI`	a `.env` file on the EC2 instance	`--env-file` at `docker run`
`AWS_REGION`, `AWS_ROLE_ARN`, `ECR_REPOSITORY`	GitHub repository **variables**	`${{ vars.X }}` in the workflow
AWS credentials	**nowhere**	minted per run by OIDC, expire in ~1 hour
ECR registry password	**nowhere**	`get-login-password` pipes it straight into `docker login`

The rules we settled on

1. **`.env` is in both `.gitignore` and `.dockerignore`.** It never enters git and never enters an image layer.
2. **Because of rule 1, `git clone` on the server does not bring it.** You must create it there or copy it with `scp`. This surprises everyone once.
3. **Anything not secret goes in a variable, not a secret.** Marking a non-secret as secret just makes logs unreadable (`***` everywhere) without adding safety.
4. **The AWS account ID is not a password**, but it is not something to publish either. Fine in the console and in your own notes; keep it out of public repositories, screenshots and forum posts.

The next step up DISCUSSED, NOT IMPLEMENTED

An `.env` file on the server works, but it is a plain-text file that anyone with shell access can read, and there is no audit trail or rotation. The standard upgrades are **AWS Secrets Manager** or **SSM Parameter Store**: the deploy script fetches the value at start-up using the instance role, so the secret is never written to disk at all. Worth doing when the app has more than one secret.

CHECK YOURSELF Â· PART 15

1. Why is it dangerous to pass `MONGODB_URI` as a `--build-arg`?
2. You add `NEXT_PUBLIC_API_KEY` to your `.env` on the server and it is `undefined` in the browser. Why?
3. You clone the repo onto a fresh server and the app cannot connect to Mongo. What is missing, and why did git not bring it?
4. Which of our four AWS-related values are genuinely secret? Justify your answer.

16 Every error we hit, and the fix

These are all real. Reading them is the fastest DevOps education in this document.

#	ERROR	REAL CAUSE	FIX
1	`usermod: group 'docker' does not exist`	Docker wasn't installed yet	Install Docker first â€” the installer creates the group
2	Permission denied on `/var/run/docker.sock`	Group change doesn't apply to an open shell	`newgrp docker`, or log out and back in
3	`UNPROTECTED PRIVATE KEY FILE`	Windows lets other accounts read the `.pem`	`icacls` â€” in two separate passes
4	`Connection closed by â€¦ port 22`	Instance out of memory or disk on a `t2.nano`	Reboot, read the system log, resize to `t3.micro`
5	`npm error code EIDLETIMEOUT`	Network stall during `npm ci` inside the build	Retry flags + cache mount; check MTU and DNS
6	Security-group rule did nothing	Source left blank and rules never saved	Fill the Source, click **Save rules**, confirm an `sgr-â€¦` ID appears
7	EC2 Instance Connect: "Error establishing SSH connection"	The browser console connects from AWS's IPs, not yours	Allow `18.206.107.24/29` on 22, or just use your own terminal
8	Provider URL rejected	Pasted over AWS's pre-filled `https://`	Clear the field, enter the URL once
9	No "Get thumbprint" button	AWS removed the step; the guide was old	Nothing to do â€” just click Add provider
10	IAM policy editor shows a red âœ—	Placeholders `<REGION>` etc. still in the ARN	Replace all three from the ECR repository URI
11	"Variable names may only contain alphanumeric characters"	A trailing space copied out of a table	Backspace once; type these names by hand
12	Pushed a branch, CI never ran	Workflow triggers only on `main` and PRs to it	Open the pull request
13	`Not authorized to perform sts:AssumeRoleWithWebIdentity`	GitHub's `sub` carries numeric IDs, not names	Put the real `sub` in the trust policy â€” see below

Error 5 in depth â€” `EIDLETIMEOUT` during the Docker build

```
1912.0 npm error code EIDLETIMEOUT
1912.0 npm error Idle timeout reached for host `registry.npmjs.org:443`
failed to solve: process "/bin/sh -c npm ci" did not complete successfully: exit code: 1
```

Reading it: the build burned 1912 seconds â€” nearly 32 minutes â€” before giving up. It is not a dependency problem. npm's connection to the registry stalled with no data arriving. Note it happened during `docker compose up` *on the EC2 box*, not on the laptop, which matters.

FIX 1

Make the install resilient â€” the Dockerfile change described in Part 5 (cache mount plus the four `--fetch-*` flags). This helps a genuinely flaky connection.

FIX 2

Check the MTU, which is the more likely root cause on EC2. AWS's ENA network interface uses MTU 9001; Docker's bridge defaults to 1500. Large TLS responses get black-holed, producing exactly this symptom: a long hang, then a timeout.

```
ip link show | grep -E '^[0-9]+:'    # the interface lines, where MTU lives
```

If the primary NIC reads 9001 and `docker0` reads 1500, force the daemon to match:

```
sudo tee /etc/docker/daemon.json <<'EOF'
{ "mtu": 1500 }
EOF
sudo systemctl restart docker
```

FIX 3

Rule out DNS with a one-line probe:

```
docker run --rm node:22-alpine npm view next version
```

A throwaway container, identical to your build environment, fetching one line of text from the registry. About 2 seconds on a healthy box. If *this* hangs, the problem is host networking and no Dockerfile edit will fix it.

THE REAL LESSON

Do not build on a tiny server at all. That single decision removes this entire class of problem â€” and it is exactly what the CI pipeline gives you.

A GREP THAT HID THE ANSWER

We first ran `ip link show | grep -E 'ens|eth|docker0'` and got only MAC-address lines. Why: `eth` matched inside the words `link/ether`, and Ubuntu's Nitro instances name the primary NIC `enX0`, which matches neither `ens` nor `eth`. A filter that quietly excludes the thing you are looking for is worse than no filter. `grep -E '^[0-9]+:'` prints the interface header lines regardless of name.

Error 13 in depth â€″ the OIDC denial SOLVED

This one took five failed builds and is the most instructive thing in the document.

SYMPTOM

```
Assuming role with OIDC
Assuming role with OIDC          # repeated a dozen times â€” the action retrying
Error: Could not assume role with OIDC:
       Not authorized to perform sts:AssumeRoleWithWebIdentity
```

WHY IT WAS HARD

AWS returns this same vague message for every possible trust failure â€” a wrong audience, a wrong subject, a role that does not exist at all. It deliberately hides which, to prevent attackers enumerating your roles. So every wrong theory produced an identical error.

WHAT WE RULED OUT, IN ORDER

1. **Wrong repo name in `sub`?** The GitHub repo is `kanban-board` while the folder and ECR repo are `kanban-app` â€” a very plausible mix-up. Checked: the trust policy already said `kanban-board`. Not it.
2. **Wrong audience?** Checked the identity provider page: `sts.amazonaws.com`. Correct.
3. **Wrong provider ARN?** Character-identical to the `Federated` line. Correct.
4. **IAM propagation delay?** Re-ran the job. Same failure.
5. **Session tags.** The debug log said `7 role session tags are being used`. Session tags need `sts:TagSession` in the trust policy. We added it â€” and also added `role-skip-session-tagging: true` to remove the variable entirely. Still failed.
6. **Wrong role ARN?** Confirmed as text from the role's own summary page â€” no `service-role/` path segment. Correct.
7. **Repo casing?** Copied the URL from the browser: `github.com/Nasir1504/kanban-board`. Matches.

Every value was genuinely correct. Checking configuration had run out of road.

THE MOVE THAT SOLVED IT

Stop guessing what AWS received; go and read what GitHub sent. We added a temporary step that decodes the run's own token and prints five harmless claims:

```
- name: Debug OIDC claims
  if: github.event_name == 'push'
  run: |
    TOKEN=$(curl -sS -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
      "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=sts.amazonaws.com" | jq -r '.value')
    PAYLOAD=$(echo "$TOKEN" | cut -d. -f2)
    PAYLOAD="${PAYLOAD}$(printf '%*s' $(( (4 - ${#PAYLOAD} % 4) % 4 )) '' | tr ' ' '=')"
    echo "$PAYLOAD" | tr '_-' '/+' | base64 -d | jq '{iss, aud, sub, repository, ref}'
```

It requests the token from GitHub's own endpoint, takes the middle section of the JWT (the payload), pads it to a valid base64 length, converts URL-safe base64 to standard base64, decodes it, and prints only `iss`, `aud`, `sub`, `repository` and `ref`. **The token itself is never printed**, and none of those five fields is a credential.

WHAT IT PRINTED

```
"sub": "repo:Nasir1504@87933341/kanban-board@1345866624:ref:refs/heads/main"
```

Against a trust policy expecting:

```
"repo:Nasir1504/kanban-board:*"
```

The repository has GitHub's **immutable identifiers** enabled, so the owner and repo carry numeric IDs. The two strings could never match. Every value we had checked in the console was correct â€” the console simply never showed us the one string that mattered.

THE FIX

One line in the trust policy:

```
"token.actions.githubusercontent.com:sub": "repo:Nasir1504@87933341/kanban-board@1345866624:*"
```

Update policy â†’ **Re-run failed jobs** â†’ green.

WE KEPT THE DEBUG STEP, COMMENTED OUT

It runs in 0 seconds and prints nothing sensitive, so it costs nothing. It now sits in `ci.yml` commented out, with a note explaining what it is for. If the trust policy ever breaks again â€” a repo rename, a new `environment:`, a tag-based trigger â€” the `sub` changes shape and this is a five-second diagnosis instead of an afternoon.

CHECK YOURSELF Â· PART 16

1. Why does AWS return the same vague message for a wrong subject, a wrong audience and a non-existent role?
2. What made the debug step succeed where seven rounds of console-checking failed?
3. Explain what the `grep` mistake was, and the general lesson about filters.
4. Two different problems can produce a hanging `npm ci`: name them, and name the one-line command that distinguishes them.

17 How to troubleshoot anything

The specific fixes above will go stale. The method will not.

Five rules

1. **Read the error's shape, not just its words.** "Times out" and "connection closed" look similar and mean opposite things: a timeout means nothing answered (firewall); a close means something answered and then gave up (the server is unwell).
2. **When one side won't say what it received, read what the other side sent.** This solved our worst bug. AWS refused to say why it rejected the token, so we printed the token's claims from GitHub's side.
3. **Shrink the failing thing until it is instant.** `npm ci` downloads hundreds of packages, so a hang tells you nothing. `npm view next version` fetches one line â€” same environment, same network, two-second answer.
4. **Change one variable at a time.** We added `sts:TagSession` *and* `role-skip-session-tagging` together; when it still failed we did not know which had been irrelevant. Not fatal here, but it costs you information.
5. **Trust text over screenshots.** A screenshot hides trailing spaces, zero-width characters, a second policy statement below the fold, and the difference between an editor view and a saved view. Paste the actual string.

Where to look, by symptom

SYMPTOM	FIRST PLACE TO LOOK	COMMAND / PAGE
CI step failed	The step's log in the Actions tab	Repo â† Actions â† the run â† the job â† expand the step
CI didn't run at all	The `on:` triggers	Does your branch or event actually match?
App not reachable	Security group, then the container	`docker ps`, then `docker logs -f kanban`
Container exits immediately	The app's own error	`docker logs kanban`
SSH hangs	Security group / your IP changed	EC2 â† Security â† Inbound rules
SSH closes	The instance is unwell	Actions â† Monitor and troubleshoot â† Get system log
Anything AWS-permission-shaped	The trust policy, then the permissions policy	IAM â† Roles â† your role â† both tabs
Disk or memory	The instance	`df -h`, `free -m`, `docker system df`

Diagnostic commands worth memorising

```
# is the network inside a container healthy?
docker run --rm node:22-alpine npm view next version

# what is this OIDC provider really? (works for any OIDC issuer)
curl -s https://token.actions.githubusercontent.com/.well-known/openid-configuration

# who am I, according to AWS?
aws sts get-caller-identity

# interface names and MTUs
ip link show | grep -E '^[0-9]+:'

# disk, memory, docker's share of the disk
df -h ; free -m ; docker system df

# the app's own output
docker logs -f kanban
```

MAKE IT FAIL ON PURPOSE

The most valuable practice, and the step nearly everyone skips. Push a commit that fails lint and watch it block the merge. Push an image that crashes at startup and notice that nothing catches it â€” then add a health check. Deploy a broken version and practise rolling back to the previous SHA tag. Break the IAM permissions deliberately and learn to read AWS's auth errors while you are calm. Each one is a real incident in miniature, and interviewers ask about them constantly.

CHECK YOURSELF Â· PART 17

1. A connection "times out" versus "is closed". What does each tell you about how far the request got?
2. State rule 2 in your own words, and give the example from our project.
3. Why is a screenshot a poor way to verify a policy?

18 Command cheat sheet

Git

```
git status --short                       # what has changed
git checkout -b feat/my-change           # new branch
git add .github/workflows/ci.yml         # stage specific files
git commit -m "ci: push image to ECR"    # save a snapshot
git push -u origin feat/my-change        # upload the branch
gh pr create --base main --fill          # open the pull request
git log --oneline -5                     # recent commits
git revert <sha>                         # undo a commit safely
```

Docker â€” on your laptop

```
docker build --platform linux/amd64 -t kanban-app:latest .
docker images
docker tag kanban-app:latest <REGISTRY>/kanban-app:latest
docker push <REGISTRY>/kanban-app:latest
docker run -p 3000:3000 --env-file .env kanban-app:latest
```

Docker â€” on the server

```
docker ps                       # running containers
docker ps -a                    # including stopped ones
docker logs -f kanban           # follow the app's output
docker pull  <REGISTRY>/kanban-app:latest
docker rm -f kanban             # stop and remove
docker run -d --name kanban --restart unless-stopped \
  -p 80:3000 --env-file .env <REGISTRY>/kanban-app:latest
docker system prune -a          # reclaim disk (careful)
docker compose up -d --build    # if you use a compose file
docker compose logs -f app
docker compose down             # stop; data safe
docker compose down -v          # stop AND DELETE VOLUMES
```

AWS CLI

```
aws sts get-caller-identity                     # who am I / account ID
aws configure                                   # set up keys + region (laptop only)
aws ecr create-repository --repository-name kanban-app --region us-east-1
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin <REGISTRY>
aws ecr describe-images --repository-name kanban-app
aws ecr put-lifecycle-policy --repository-name kanban-app \
  --lifecycle-policy-text file://ecr-lifecycle.json
```

SSH & the server

```
# Windows: fix key permissions (PowerShell)
icacls "path\to\key.pem" /inheritance:r
icacls "path\to\key.pem" /grant:r "$($env:USERNAME):(R)"

ssh -i "path\to\key.pem" ubuntu@<PUBLIC-IP>     # Ubuntu AMI
ssh -i "path\to\key.pem" ec2-user@<PUBLIC-IP>   # Amazon Linux AMI
ssh -v -i ...                                    # verbose, for diagnosing
scp -i "path\to\key.pem" .env ubuntu@<IP>:~/    # copy a file up

df -h ; free -m                # disk and memory
sudo systemctl status docker   # is Docker running
newgrp docker                  # apply the group without logging out
```

SAVE YOURSELF THE TYPING

Add this to `C:\Users\<you>\.ssh\config`, then the whole command is just `ssh kanban`:

```
Host kanban
  HostName <PUBLIC-IP>
  User ubuntu
  IdentityFile C:/Users/NasirPC/.ssh/nasir-ec2-server.pem
```

GitHub Actions expressions

EXPRESSION	MEANING
`${{ vars.NAME }}`	A repository variable (readable)
`${{ secrets.NAME }}`	A repository secret (masked in logs)
`${{ steps.<id>.outputs.<key> }}`	An output from an earlier step
`${{ github.event_name }}`	`push` or `pull_request`
`${{ github.ref }}`	e.g. `refs/heads/main`
`$GITHUB_SHA`	The commit hash, as a shell variable

19 Interview questions

Every one of these is answerable from something we actually did. Answers are given inline here â€” these are for rehearsal, not self-testing.

1. What is the difference between CI and CD?

CI verifies every change automatically â€” install, format, lint, build. CD takes a verified change onward: continuous *delivery* produces a ready-to-release artifact automatically and a human releases it; continuous *deployment* releases it automatically too. Our pipeline builds and publishes a Docker image on every merge to `main`, and a person runs the deploy â€” that is continuous delivery.

2. What is the difference between an image and a container?

An image is a frozen, read-only filesystem plus a start command. A container is a running instance of one, with a thin writable layer on top that is discarded when the container is removed. One image, many containers.

3. Why use a multi-stage Docker build?

Because build-time needs and run-time needs are different. Our build stage needs the compiler, dev dependencies and the whole source tree; the runtime needs only `.next/standalone`, the static assets and `public`. Multi-stage lets you build fat and ship thin â€” smaller images, faster pulls, and a smaller attack surface because build tools and source never reach production.

4. Why `npm ci` rather than `npm install` in a build?

`npm ci` deletes `node_modules`, installs exactly what `package-lock.json` specifies, never mutates the lockfile, and fails if `package.json` and the lockfile disagree. Builds must be reproducible; `npm install` can quietly resolve different versions.

5. How does your CI authenticate to AWS, and why that way?

OIDC, not access keys. GitHub mints a short-lived signed token for each run describing the repo and branch. An IAM role's trust policy checks the issuer, audience and subject, then AWS returns credentials that expire in about an hour. Nothing secret is stored in the repository, so nothing can leak, and rotation is automatic.

6. What is the difference between a trust policy and a permissions policy?

The trust policy answers *who may assume this role*; the permissions policy answers *what they may do once they have*. Our trust policy admits GitHub Actions runs from one specific repository; the permissions policy allows pushing images to one specific ECR repository.

7. What is the most important line in an OIDC trust policy?

The `sub` condition. The issuer URL is shared by every repository on GitHub, so without a subject condition *any* repository on the internet could assume your role. The `sub` is the only thing that scopes it to yours.

8. Why tag images with the commit SHA as well as `latest`?

`latest` is a moving pointer â€” useful for "deploy the newest thing", useless for knowing what is actually running. The SHA tag is immutable: it will always mean exactly that commit. That is what makes rollback a one-command operation and makes an incident traceable to a diff.

9. Explain least privilege with an example from your project.

Three ECR policies exist: ReadOnly, PowerUser and FullAccess. The CI role only needs to push, so FullAccess â€” which can delete repositories â€” is wrong. The EC2 instance only needs to pull, so it gets ReadOnly. The tightest version is an inline policy naming

a single repository ARN, with `ecr:GetAuthorizationToken` on `*` because AWS requires it at account level.

10. Why doesn't a pull request push an image?

Because the code isn't approved yet. Publishing from a PR would put unreviewed code in the registry and overwrite `latest`. We gate the AWS steps with `if: github.event_name == 'push'` and keep a PR-only `docker build` so the "does it build?" signal survives.

11. Where do secrets live in a containerised deployment?

Never in the image. Build-time arguments are baked into layer history and readable by anyone who pulls the image. Runtime environment variables â€" `--env-file` or `-e` â€" are supplied when the container starts. The exception is `NEXT_PUBLIC_*` values, which must be present at build time and are public by definition, so they must never be secret.

12. Jenkins or GitHub Actions?

They answer the same four questions with different syntax. Actions is managed, so there is no server to patch or back up, and it is the obvious choice when the code already lives on GitHub. Jenkins wins for air-gapped or regulated environments, exotic build hardware, or an existing investment. The trade-off Actions asks you to accept is lock-in to GitHub â€" real, but small, since CI configs translate mechanically.

13. Tell me about a CI/CD problem you debugged.

A perfect answer already exists in Part 16: the OIDC denial. Structure it as â€" symptom, why the error was uninformative, the hypotheses ruled out, the decision to stop checking configuration and instead print what GitHub was actually sending, the discovery that immutable identifiers put numeric IDs in the `sub`, the one-line fix, and the debug step kept commented out for next time.

14. How would you make this pipeline production-ready?

In order: add automated tests as a gate; automate the deploy via SSM so there is no manual step; add a health check so a crashing image is noticed; add a staging environment fed from a `dev` branch; move secrets from a file on disk into Secrets Manager or Parameter Store; eliminate the downtime gap between `docker rm` and `docker run`; and define the AWS resources in Terraform so the console clicking becomes reproducible.

15. What is the difference between EBS and Elastic Beanstalk?

EBS is Elastic Block Store â€" virtual disks attached to EC2 instances; the instance's root volume is one. Elastic Beanstalk is a managed platform that deploys applications for you. The names are similar and unrelated.

20 Answers to the understanding questions

Cover this page while you work through the "Check yourself" boxes.

Part 1 Â· What CI/CD means

1. Delivery = every good commit becomes a ready-to-release artifact, and a human triggers the release. Deployment = it goes live automatically. We do **delivery**: the image lands in ECR automatically; running it is manual.
2. Question 2 â€” the gates. They decide whether the pipeline is allowed to proceed.
3. Only that the checks we wrote passed: formatting, lint, and that the Docker image builds. It proves *nothing* about whether the app behaves correctly â€” we have no tests. Green means "not obviously broken", not "correct".

Part 2 Â· Architecture

1. `next build` stops producing `.next/standalone`, so the Dockerfile's `COPY --from=builder /app/.next/standalone ./` fails. You would notice at the `docker build` step in CI, not in local development.
2. The `-p 80:3000` flag on `docker run`. Docker forwards traffic from the host's port 80 to port 3000 inside the container.
3. `239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app` â€” the AWS account ID, the region, and the repository name (plus the fixed `dkr.ecr` service hostname).

Part 3 Â· Git & GitHub

1. No. The workflow triggers on push to `main` and on pull requests *targeting* `main`. A plain branch push matches neither. Open a PR and it runs.
2. The code isn't reviewed or approved. A PR push would put unmerged code in the registry and overwrite the `latest` tag that the server pulls.
3. Because the ARN is only an address, not a credential â€” knowing it grants nothing. Only a valid OIDC token from our specific repository can assume that role. It would need to be a secret if it were paired with something that *did* grant access, such as an access key.

Part 4 Â· Docker ideas

1. An image is the frozen recipe result sitting in the freezer â€” read-only, identical everywhere. A container is what you get when you start one: a live process with its own filesystem and network, plus a scratch layer that is thrown away when it is removed.
2. Your open shell still holds the group list it had at login. Fix it with `newgrp docker`, or log out and SSH back in.
3. They are gone. Writes went into the container's disposable top layer. To keep data you need a named volume or a bind mount.

Part 5 Â· Dockerfile

1. Layer caching. That layer is invalidated only when the dependency files change, so editing a component does not force a full `npm ci` on every build.
2. `npm ci` wipes `node_modules`, installs exactly the locked versions, never edits the lockfile, and fails if the two files disagree. CI needs that determinism; `npm install` may resolve different versions on different days.
3. Next.js compiled it. `.next/standalone` contains a generated `server.js` and only the JavaScript actually needed at runtime â€” the source is no longer required.
4. (a) The container isn't publishing the port â€” no `-p 3000:3000` on `docker run`. (b) The security group has no inbound rule for 3000. A third possibility: the app bound to `localhost` inside the container, needing `-e HOSTNAME=0.0.0.0`.

Part 6 Â· Build & tag

1. Because the registry destination is encoded in the image's *name*. Docker reads the hostname portion of the tag to decide where to send it; there is no separate destination argument.
2. Immutability. `latest` changes meaning with every push, so it cannot tell you what is running or let you go back. The SHA tag always means one exact commit.
3. `t4g` is AWS Graviton â€” an ARM processor. The image was built for x86-64, so the binary format is wrong. Rebuild with `--platform linux/arm64`, or use an x86 instance type.

Part 7 Â· Containers

1. Host first, container second: 80 is the server's port, 3000 is inside the container. Swapped, you would be publishing the server's port 3000 to the container's port 80 â€” where nothing is listening â€” so the site would not load.
2. A container's writable layer is deleted with the container. A volume stores the data on the host so it survives. `down` removes containers only; `down -v` also deletes the named volumes, erasing the database.
3. Inside a container, `localhost` means that container itself, not the host and not a sibling. Use the Compose service name (`mongo`), which the built-in DNS resolves to the right container.
4. The app silently staying down after a crash or a server reboot. Docker restarts it automatically unless you stopped it deliberately.

Part 8 Â· GitHub Actions

1. It downloads your repository onto the runner. Without it the machine is empty and every later step fails â€” there is no `package.json` to install and no Dockerfile to build.
2. It allows the job to request a short-lived OIDC identity token from GitHub. Without it the AWS step fails with "Credentials could not be loaded."
3. One is the PR-only check, one is inside the push-only build-and-push step. Their `if:` conditions are mutually exclusive, so exactly one runs in any given job.
4. From the `amazon-ecr-login` action, which is given the `id: ecr`. It is better than hard-coding because the account ID and region come from whatever credentials the job assumed â€” no value to keep in sync, and nothing account-specific committed to the repo.

Part 9 Â· EC2

1. The instance type is the hardware shape â€” vCPUs, RAM, network â€” with no operating system. The AMI is the disk image, and it decides the username: Ubuntu â† `ubuntu`, Amazon Linux â† `ec2-user`.
2. A hang means nothing answered: the security group is blocking port 22, or your home IP changed. `Permission denied (publickey)` means the server *did* answer and rejected your identity â€” wrong username for the AMI, or the wrong key file.
3. `t3.micro` has 1 GB rather than 0.5 GB **and** is free-tier eligible, whereas `t2.nano` costs about $4/month. More RAM for less money.
4. An unattached EBS volume left in state `available`, and an Elastic IP not associated with a running instance (about $3.60/month). Snapshots too.

Part 10 Â· ECR

1. The GitHub runner is a rented machine that is destroyed after the run. The image on it disappears with it. A registry is the durable, shared place both the runner and the server can reach.
2. Account ID `239068269281`, region `us-east-1`, repository `kanban-app`, tag `latest`.

3. Because the EC2 instance carries an IAM role, and the AWS CLI picks up temporary credentials from the instance metadata automatically. Docker Hub has no concept of AWS identity, so you would have to store a username and access token on the box.
4. You lose your rollback, your ability to launch a replacement or second server, and your only recovery from `docker system prune -a`. The saving is roughly three cents a month.

Part 11 Â· EBS & storage

1. EBS = Elastic Block Store, the virtual disks attached to EC2 â€” we used it as the instance's root volume. Elastic Beanstalk is a separate managed deployment platform we never touched.
2. The EBS volume, at roughly $0.08 per GB per month. Only the compute charge stops.
3. `sshd` needs to write session and PAM files to start a login. With no space it cannot, so it accepts the TCP connection and then immediately drops it.
4. On the instance's EBS volume, under `/var/lib/docker/volumes/<project>_<name>/_data`.

Part 12 Â· OIDC

1. The trust policy is the lock on the locker â€” who may put the uniform on. The permissions policy is the badge on the uniform â€ which doors that person can then open.
2. The subject: which repository, branch or context the run belongs to. Omit it and the only remaining condition is the audience, which every GitHub repository on earth satisfies â€ so any repository could assume your role.
3. It is an account-level API call, not scoped to a repository. AWS rejects the statement if you try to pin it to a repository ARN. It is a quirk of that specific action.
4. The signed OIDC token. GitHub signs it, AWS verifies the signature against the registered identity provider, and then matches its claims against the trust policy conditions. Possession of the token for one run is the proof.
5. It is denied. When a job declares an environment, the `sub` becomes `repo:OWNER/REPO:environment:production` â€ the branch segment disappears entirely, so a branch-pinned condition can never match.

Part 13 Â· Deployment

1. Different consumer and different permission. The GitHub role is assumed by an external identity via OIDC and may push; the instance role is worn by the machine and may only pull. Sharing one role would give the server push rights it does not need.
2. Least privilege. If the instance is compromised, ReadOnly cannot overwrite or delete your images. CI genuinely needs to push, so it gets PowerUser (or better, a repo-scoped inline policy).
3. The ECR login token expired â€ it lasts 12 hours. Re-run the `aws ecr get-login-password â€¦ | docker login â€¦` line.
4. `./deploy.sh <previous-commit-sha>`. The script's `${1:-latest}` takes the tag as its first argument, and every build pushed a SHA tag for exactly this purpose.

Part 14 Â· The whole pipeline

1. Merge creates a commit on `main` â†’ the `push` trigger fires â†’ runner checks out, installs, formats, lints â†’ GitHub mints an OIDC token â†’ AWS STS validates it against the trust policy and returns temporary credentials â†’ `amazon-ecr-login` runs `docker login` â†’ the image is built and tagged twice â†’ both tags are pushed to ECR.
2. Because the only thing that mattered â€ the built image â€ was pushed to ECR before the runner was destroyed. Everything else on the runner was disposable by design.
3. GitHub repository secrets (replaced by OIDC), the workflow file itself (replaced by `vars`, which hold no credentials), and the EC2 instance (replaced by an IAM instance role). The only long-lived secret anywhere in the project is the SSH private key on

your laptop.

Part 15 Â· Environment variables

1. Build arguments are recorded in the image's layer history. Anyone who can pull the image can read them, and the image sits in a registry.
2. `NEXT_PUBLIC_*` values are compiled into the browser bundle at build time. Supplying it at run time is too late â€” the JavaScript has already been generated.
3. The `.env` file. It is in `.gitignore`, so git never tracked it, and in `.dockerignore`, so the image does not contain it either. Create it on the server or copy it with `scp`.
4. None of them. The region is public knowledge, the ECR repository name is meaningless without access, and the role ARN is an address that only a valid OIDC token from our repository can act on. That is precisely why OIDC was worth the extra setup.

Part 16 Â· Errors

1. Deliberate design: a specific message would let an attacker probe which role names exist and which conditions they use. AWS trades your debugging convenience for that protection.
2. The console showed us what we had *configured*; it never showed what GitHub was actually *sending*. Decoding the token's claims revealed the numeric immutable identifiers in the `sub`, which nothing in the AWS console could have told us.
3. `grep -E 'ens|eth|docker0'` matched `eth` inside `link/ether` and missed the real interface, named `enX0`. Lesson: a filter that quietly excludes what you are looking for is worse than no filter â€” widen it before trusting a negative result.
4. A genuinely flaky network, or an MTU mismatch between the EC2 interface (9001) and Docker's bridge (1500). Distinguish them with `docker run --rm node:22-alpine npm view next version`: instant means the network is fine, hanging means it is host-level.

Part 17 Â· Troubleshooting

1. A timeout means nothing ever answered â€” the packet was dropped, usually by a firewall or security group. A close means something answered and then gave up, so the network path is fine and the service itself is unwell.
2. When one side refuses to say what it received, print what the other side sent. AWS would not say why the token was rejected, so we decoded the token GitHub was issuing and compared the real `sub` to the policy.
3. It hides trailing spaces, invisible characters, content below the fold such as a second policy statement, and whether you are looking at a saved value or an unsaved editor. Paste text.

21 Status summary & what comes next

What is actually built

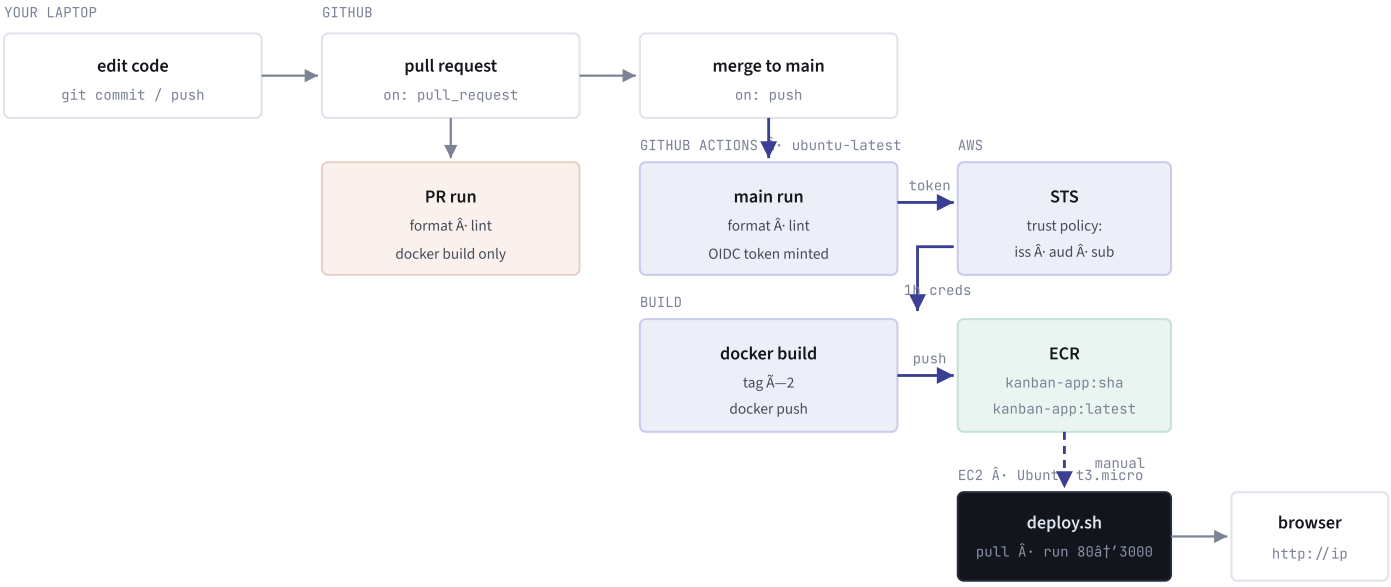
THING	STATUS	WHERE
Multi-stage Dockerfile with hardened `npm ci`	IMPLEMENTED	Part 5
`.dockerignore` excluding `.env` and `*.pem`	IMPLEMENTED	Part 5
CI: format check, lint, Docker build on PRs	IMPLEMENTED	Part 8
GitHub OIDC identity provider in AWS	IMPLEMENTED	Part 12
IAM role `github-actions-ecr-push` + trust policy	IMPLEMENTED	Part 12
ECR repository `kanban-app`	IMPLEMENTED	Part 10
CI pushes `:sha` + `:latest` on merge to main	IMPLEMENTED	Part 8
EC2 instance, security group, SSH access	IMPLEMENTED	Part 9
Docker installed on EC2	IMPLEMENTED	Part 4
Commented-out OIDC debug step, kept for next time	IMPLEMENTED	Part 16
EC2 instance role `ec2-ecr-pull` + pull & run	NEXT UP	Part 13
`deploy.sh` on the instance	NEXT UP	Part 13
Automated deploy via SSM or SSH action	DISCUSSED	Part 13
ECR lifecycle policy	DISCUSSED	Part 10
Docker Compose with MongoDB + named volume	DRAFTED, NOT COMMITTED	Part 7
Swap file on the instance	DISCUSSED	Part 9
Secrets Manager / SSM Parameter Store	DISCUSSED	Part 15
Automated tests, staging, health checks, HTTPS	NOT STARTED	Part 13
Jenkins	EXPLAINED ONLY	Part 8

A learning order that works

1. **Close the loop.** Finish the EC2 side by hand, at least three times. The tedium teaches you what a deploy is, so automating it later is not cargo-culting.
2. **Break it on purpose.** Fail lint. Ship a crashing image. Roll back. Revoke an IAM permission and read the error while calm. Each is a real incident in miniature.
3. **Add the missing gates.** Tests first â€” even three trivial ones, because the point is wiring `npm test` into the pipeline. Then a staging environment on a `dev` branch. Then proper secret storage. Then zero-downtime deploys.
4. **Then read theory**, once it maps onto scars you have: trunk-based development, blue/green and canary deploys, Infrastructure as Code (Terraform â€” it turns all your console clicking into a reproducible file), and observability.

5. **Skip Kubernetes for now.** It is the loudest thing in this space and will eat months if you reach for it before the fundamentals are solid. Learn it when a job requires it.

The complete flow, one picture



SOLID BLUE = AUTOMATED TODAY. DASHED = STILL MANUAL (PART 13).

â˜… One-page revision

The four questions every pipeline answers

1. When? push to `main`, or PR to `main`. **2. What gates?** prettier, eslint, docker build. **3. What artifact?** a Docker image tagged `:sha` and `:latest`. **4. How does it ship, and how do I undo it?** pull and run on EC2; roll back by running the previous SHA tag.

The chain, in one line

`edit â†’ commit â†’ PR (checks) â†’ merge â†’ CI (checks + OIDC + build + push) â†’ ECR â†’ pull on EC2 â†’ docker run -p 80:3000 â†’ browser`

Six definitions

Image	frozen filesystem + start command, read-only
Container	a running image with a disposable writable layer
Registry (ECR)	where images live between build and run
Trust policy	WHO may assume this IAM role
Permissions policy	WHAT they may do once they have
Security group	the firewall â€” nothing gets in unless a rule says so

Ten commands

```
git push -u origin my-branch          gh pr create --base main --fill
docker build --platform linux/amd64 -t app .
docker push <REGISTRY>/kanban-app:latest
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
<REGISTRY>
docker pull <REGISTRY>/kanban-app:latest
docker run -d --name kanban --restart unless-stopped -p 80:3000 --env-file .env <IMAGE>
docker logs -f kanban                 docker ps
ssh -i key.pem ubuntu@<IP>            df -h ; free -m
```

Five rules that prevent most pain

1. **Never build on a tiny server.** Build in CI; the server only pulls and runs.
2. **Never bake a secret into an image.** Runtime `--env-file`, always.
3. **Always tag with the commit SHA.** It is your rollback and your audit trail.
4. **Give every identity the smallest permission that works.** CI pushes, the server reads.
5. **When one side won't say what it received, print what the other side sent.**

The five errors you will meet again

`group 'docker' does not exist`	install Docker first
`UNPROTECTED PRIVATE KEY FILE`	`icacls /inheritance:r` then `/grant:r`
SSH hangs / SSH closes	firewall / the instance is out of RAM or disk
`Not authorized to â€¦ AssumeRoleWithWebIdentity`	print the real `sub` and match it
`exec format error`	wrong CPU architecture â€” check `uname -m`

Your next three moves

1. Create the `ec2-ecr-pull` instance role and attach it. **2.** Pull and run the image by hand, three times, until it is boring. **3.** Put those four commands in `deploy.sh`.

Compiled 2 September 2026 from the CI/CD conversations for Nasir1504/kanban-board.
Real values in this document â€” AWS account 239068269281, the ECR address, the instance details â€” are yours.
Keep this file out of public repositories and screenshots.